A Major Project Report
On

LORA BASED SOLDIER SECURITY SYSTEM WITH INTEGIRATED IOT

Submitted for partial fulfilment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING

BY

Ms. N. HARIKEERTHANA (22UK1AO4A7)

Under the Guidance of
Dr. K. Manasa
Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VAAGDEVI ENGINEERING COLLEGE

(An Autonomous Institution | Affiliated to JNTU Hyderabad)
Accredited by NAAC with 'A+' Grade, Certified by ISO 9001:2015 Approved by AICTE, New Delhi

2025-2026

VAAGDEVI ENGINEERING COLLEGE

(An Autonomous Institution | Affiliated to JNTU Hyderabad)

Accredited by NAAC with 'A+' Grade, Certified by ISO 9001:2015 Approved by AICTE, New Delhi

Bollikunta, Warangal-506 005, Telangana, India

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the project work entitled "**LORA BASED SOLDIER SECURITY SYSTEM WITH INTEGIRATED IOT"** is a Bonafide work carried out by **Ms.N.HARIKEERTHANA(22UK1A7)** in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in ***Electronics and Communication Engineering*** from Vaagdevi Engineering College, (Autonomous) during the academic year 2025 - 2026.

Dr. K. Manasa	**Dr. K. Naveen**	**Dr. V. Manohar**
Assistant Professor	**Assistant Professor**	**Associate Professor**
Project Guide	**Project Coordinator**	**Head of the Department**

DECLARATION

I declare that the work reported in the project entitled "**LORA BASED SOLDIER SECURITY SYSTEM WITH INTEGIRATED IOT"** is a record of work done by us in the partial fulfilment for the award of the degree of **Bachelor of Technology** in **Electronics & Communication Engineering, VAAGDEVI ENGINEERING COLLEGE** (An Autonomous Institution & Affiliated to JNTU Hyderabad) Accredited by NAAC with 'A+' Grade, Certified by ISO 9001:2015 Approved by AICTE, New Delhi, Bollikunta, Warangal-506 005, Telangana, India under the guidance of **Dr. K.MANASA,** Assistant Professor**,** ECE Department, I hereby declare that this project work bears no resemblance to any other project submitted at Vaagdevi Engineering College of or any other university/college for the award of the degree.

Ms. N. HARIKEERTHANA (22UK1A04A7)

ACKNOWLEDGEMENT

The development of the project though it was an arduous task, it has been made by the help of many people. I pleased to express our thanks to the people whose suggestions, comments, criticisms greatly encouraged us in betterment of the project.

I would like to express my sincere gratitude and indebtedness to my project Guide **Dr.K.MANASA,** Assistant Professor**,** for his/her valuable suggestions and interest throughout the course of this project.

I am also thankful to Head of the Department **Dr. V.Manohar**, **Associate Professor** for providing excellent infrastructure and a nice atmosphere for completing this project successfully.

I would like to express my sincere thanks and profound gratitude to **Dr.M.Shashidhar**, principal of **Vaagdevi Engineering College,** for his support, guidance, and encouragement in the course of our project.

I am also thankful to Project Coordinator **Dr.K.Naveen,** Assistant Professor**,** for their valuable suggestions, encouragement and motivations for completing this project successfully.

I am thankful to all other faculty members for their encouragement.

I convey my heartfelt thanks to the lab staff for allowing me to use the required equipment whenever needed.

Finally, I would like to take this opportunity to thank my family for their support through the work. I sincerely acknowledge and thank all those who gave directly or indirectly their support in completion of this work.

Ms. N. HARIKEERTHANA (22UK1A04A7)

CONTENTS

ABSTRACT

The "LoRa-Based Soldier Security System Integrated with IoT" aims to enhance real-time monitoring and safety of soldiers in the field through advanced wireless communication and IoT technologies. The system comprises two key units: a Transmitter (Tx) and a Receiver (Rx). The Tx unit integrates vital sensors such as SpO2 and heartbeat sensors, a keypad emergency button, an LCD display, a buzzer, and a LoRa transmitter module. The Rx unit consists of a LoRa receiver module, an LCD display, a buzzer, and IoT integration for remote monitoring. Data from the sensors on the Tx side, including critical health parameters, are continuously monitored. If any parameter crosses predefined threshold values or the emergency button is pressed, an alert is triggered on both the Tx and Rx units via buzzers and LCD displays. Simultaneously, the alert is transmitted to an IoT server for real-time notification to higher authorities. The system employs an Arduino Uno on the Tx side for sensor data acquisition and an ESP32 on the Rx side for IoT communication. This project offers a reliable, low-power, and long-range communication solution, ensuring timely responses during critical situations and improving overall soldier safety and operational efficiency.

LIST OF FIGURES

LIST OF TABLES

Name of the Table **Page No**.

CHAPTER – 1

INTRODUCTION

An embedded system is a special-purpose computer system designed to perform one or a few dedicated functions, sometimes with real time computing constraints. It is usually embedded as part of a complete device including hardware and mechanical parts. In contrast, a general purpose computer, such as a personal computer, can do many different tasks depending on programming. Embedded systems have become very important today as they control many of the common devices we use. Since the embedded system is dedicated to specific tasks, design engineers can optimize it, reducing the size and cost of the product, or increasing the reliability and performance. Some embedded systems are mass produced, benefiting from economies of scale. Physically embedded systems range from portable devices such as digital watches and MP3 players to large stationary installations like traffic lights, factory controllers, or the systems controlling nuclear power plants. Complexity varies from low, with a single microcontroller chip, to very high with multiple units, peripherals and networks mounted inside a large chassis or enclosure. In general, "embedded system" is not an exactly defined term, as many systems have some element of programmability. For example, Handheld computers share some elements with embedded systems — such as the operating systems and microprocessors which power them — but are not truly embedded systems, because they allow different applications to be load and peripherals to be connected. An embedded system is some combination of computer hardware and software, either fixed in capability or programmable, that is specifically designed for a particular kind of application device. Industrial machines, automobiles, medical equipment, cameras, household appliances, airplanes, vending machines, and toys (as well as the more obvious cellular phone and PDA) are among the myriad possible hosts of an embedded system. Embedded systems that are programmable are provided with a programming interface, and embedded systems programming is a specialized occupation. Certain operating systems or language platforms are tailored for the embedded market, such as Embedded Java and Windows XP Embedded. However, some low-end consumer products use very inexpensive

The soldier Health and Position Tracking System allows military to track the current GPS position of soldier and also checks the health status including body temperature and heartbeats of soldier. The System also consists extra feature with the help of that soldier can ask for help manually or send a distress signal to military if he is in need. The GPS modem sends the latitude and longitude position with link pattern with the help of that military can track the current position of the soldier. The system is very helpful for getting health status information of soldier and providing them instant help.

In this system, smart sensors are attached to the body of soldiers. As soon as any other soldier enters the enemy lines it is very difficult for the army base station to know about the location as well as the health status of all soldiers. The important and vital role is played by the soldiers. This system will be useful for soldiers,

who involve in missions or in special operations. This personal server will provide the connectivity to the server at the base station using a wireless connection. There are many concerns regarding the safety of these soldiers. The M-health can be defined as mobile computing, medical sensors and communication technologies for health care. Each soldier also has a GSM (Global system for Mobile communication) module which enables the communication with the base station in case of injuries. In today's era enemy warfare is an important factor in any nation's security. The national security mainly depends on army (ground), navy (sea), air-force (air). In our project we have come up with an idea of tracking soldier as well as to give status of the soldier during the war. It is possible by M-Health. The defence department of country must be effective for the security of that country. This is implemented with a personal server for complete mobility. This system enables GPS (Global positioning systems) tracking of these soldiers.

A portable, wireless low-cost tracking system with high reliability is the need of hour for the protection of valuable life of the soldiers on the battle fields. Further, soldiers can be guided for the correct directions during the operations using GPS. The army suffers a lot due to the unavailability of information of injuries to its personnel which may increase the death/ permanent disability toll. The armed forces when deployed in battlefield need to be monitored for better utilization of soldiers and using strategies to man oeuvre them to combat. It is observed that the causalities are caused due to injuries rather than the direct assaults in the battlefield. This number can be minimized if the real-time information is available at the control room about the health and location of the soldier. There are many issues regarding the safety of soldiers. The transmission of these parameters to the control room is carried out by the control room receives the position and orientation of soldier from GPS.

Pulse rate, body temperature, and oxygen level in an environment can be monitored along with the location tracking of the soldiers using GPS can be monitored using the proposed system. However, all these technologies suffered from one or more reasons like high installation cost, loss of signal, high noise as well as the bulky nature. Knowledge of current location of soldiers, inability of continuous communication with the control room during the operations, lack of immediate medical attention and operations under different geographical conditions are the few prominent safety issues. Indian armed forces are the third largest standing army in the world with 1,200,255 active troops and 990,960 reserve troops. Rather than reporting on the status of their health and current location which may sometimes be wrong due to human error, a device attached with sensors can be used to get precise results.

CHAPTER – 2
LITERATURE REVIEW

Our objective was to establish a cost-effective and consistent project that would aid the base unit in terms of soldier health and security during wartime special operations. Furthermore, soldiers can submit requests for assistance to the base Station. At first obtaining the physical parameters like body temperature, heartrate and oxygen level of the soldier's body [1].

Then Tracking the location of the soldier through GPS. After that obtaining the environmental factors of soldier like atmospheric temperature and atmospheric pressure. Data obtained in these cases is processed through the blynk server and displaying the information in the blink app. If any abnormalities found in the data obtained from the soldier the soldier, Alerting the soldier and authority in emergency [2].

During the conflict, this project has an associated implementation of tracking the soldier and navigating between soldiers, such as getting their speed, distance, and health state, which allows military decision makers to put up war strategies. As a result, they can take immediate action by directing help to soldiers who have requested it. Soldiers' health constraints are monitored using a variety of biological sensors, and their location and placement are restricted using a GPS module [3].

ZigBee and GSM wireless technology were used to send current updates of patients to the doctor and then doctors can take immediate action against that patient. So ATmega328P better than other processors. Data originating from sensors and GPS receiver is processed and collected using Arduino (ATmega328P) processor. AT89C51 microcontroller was used to collect health parameters and then these parameters are transferred through GSM to the base unit. LM35 temperature sensor, Pulse Rate sensor and oxygen level detector sensor for continuously monitoring health status of soldier [4].

Many efforts were reported by different academicians and researchers to track the location of the soldiers' along with their health condition on the battlefield Pavan Kumar et.al.The base station can access the current status of the soldier using IOT as the different tracking parameters of the soldier get transmitted via Wi-Fi module. reporteda GPS based technology to monitor the soldier health parameters and location tracking using GPS. Jassaz al .proposed an idea of integration of wireless sensor network and cloud computing for the information processing in real-time and speedy manner. A ZigBee based approach was proposed in the collected information were then added to the cloud-based websites with the help of IoT [5].

A wireless body area sensor networks (WBASNs)technology using ZigBee was reported in to continuously monitor the human health and its location. A real-time, ARM processor based approach for the monitoring and collection of temperature, heartbeat, ECG parameters of patients by GPS is used to determine realtime position and orientation [6].

A Google map based approach was proposed in to track the location of the soldiers. These information will be stored on the Cloud and can be extracted on the PC of control room, as and when extracted. Using various biomedical sensors, health parameters of a soldier is observed along with its surrounding environment condition observed [7].

The proposed system is divided into two unit i.e. of the GPS to guide the soldier in correct direction. Based on these information, the authorities can initiate immediate action by deploying a medical, rescue team or any backup force for their help. RF based module to gather the live information of soldiers on the battlefield was proposed by [8].

G. Shaikh. The specific choice of processor is due to the facts that, as compared to the other data possessors used in existing system; Arduino board is a low cost and easily available with flexible interfacing capability. Hence, a portable wireless real-time system based on IoT concept is developed and proposed in this paper which will be an effective alternative to the existing technologies in the area of soldiers' health and location tracking on the battlefields. Soldier unit and control room unit. A Raspberry Pibased approach was proposed in to monitor the body temperature, respiration, movements and heartbeat ofthe patient. Further, a one-timepassword (OTP) based system was proposed in to secure and authenticate the data processing provides the state-of-the art soldiers health and location monitoring system. However, all these systems are stuck-up by one or more reasons like costly implementation, delay in response and bulky nature [9].

2.1: The Technology A robust accurate positioning system with seamless indoor and outdoor coverage is highly needed tool for increasing safety in emergency response and military operation. GPS-based positioning methods mainly used to field rescue. The position and orientation of the rescuer and the trapped is acquired using GPS chip [10].

Using the GPS data of both the units the relative distance, height and orientation between them are calculated from the geometric relationships based on a series of formulas in Geographic Information Science (GIS). Using this technology, we are doing the navigation between two soldier .the data will be sent wirelessly by RF Transceiver. This device can do accurate coordination via wireless communication, helping soldier for situational awareness. GPS module have serial interface. Receiver information are broadcast via this interface in a special data format. This format standardized by the National Marine Electronics Association (NMEA). 2.2: Evaluation The "soldier tracking and health monitoring system" is an effective security and safety system which is made by integrating the advancements in wireless and embedded technology. It helps for a successful secret mission [11].

CHAPTER – 3

EXISTING SYSTEM

In the existing system they have used Arduino board and the microcontrollers which are different from the microcontroller that we have used. For transferring the data to the authorities and communication between the soldiers near to them and the authorities or base station they have used Zigbee module, Lora WAN module and GSM module. In the existing system they measured the parameters like body temperature, heart rate and atmospheric temperature of the soldier. While using the Arduino we have to use Analog to Digital converters externally. Existing methods include traditional methods of communication such as walkie-talkies. Other methods include location tracking through the use of other controllers such as : PIC18F452 Microcontroller , Node MCU, These existing systems offer location tracking with the help of GPS modules.

CHAPTER – 4

PROPOSED SYSTEM

The block diagram of GPS based soldier tracking and health indication system is shown in fig. it consist of two units soldier unit and base station unit. As it requires high speed communication it is intended to use Arduino controller which is based on with real-time emulation and embedded trace support, that combines the 2GB of embedded high speed Flash memory. Biosensors such as Body temperature and pulse rate are integrated to Raspberry Pi processor to monitor the health status. The GPS receiver is used to log the longitude and latitude of soldier, which is stored in microcontroller memory. GPS Receiver receives and compares the signal from orbiting GPS satellite to determine geographic position.

TX BLOCK:

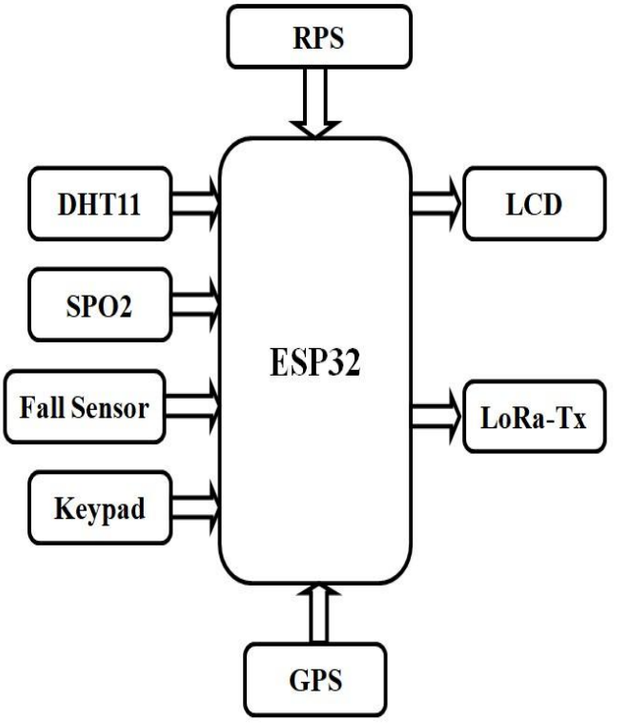


Fig:4.1: LORA Transmitter Block Diagram

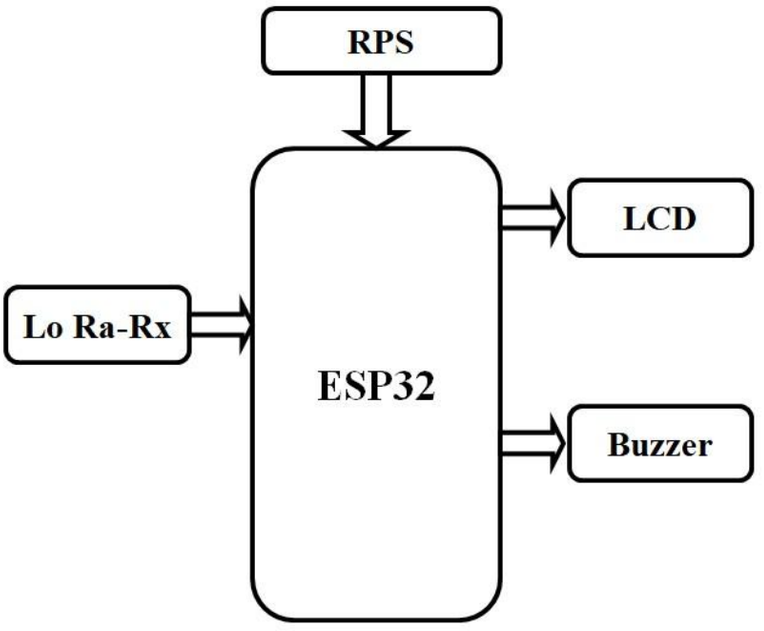


Fig 4.2: LORA Receiver Block Diagram

This is a block diagram referring soldier security using embedded electronics .Our project is characterized as five sections, which are RPS section, input section, output section, microprocessor section and programming section. A regulated power supply (RPS) is an embedded circuit, used to convert unregulated alternating current into a stable direct current by using a rectifier. The main function of this is to supply a constant voltage to a circuit that should be functioned in a particular power supply limit. Nour project we are supplying 5V to the raspberry pi board.

The input equipment of our project are temperature sensor which is DHT11. The DHT-11 Digital Temperature and Humidity Sensor is a basic, ultra low-cost digital temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air and spits out a digital signal on the data pin (no analog input pins needed). The input section also consists of heart beat sensor.Heartbeat sensor is designed to give digital output of heat beat when a finger is placed on it. When the heart beat detector is working, the beat LED flashes in unison with each heartbeat. This digital output can be connected to raspberry pie directly to measure the Beats Per Minute (BPM) rate. It works on the principle of light modulation by blood flow through finger at each pulse.Theinputs temperature and heart beat are sent to the processor section.

Our project output consists of LCD display which shows the latitude and longitude of a soldier's region which captured by the GPS receiver.The temperature and heartbeat are displayed on the LCD display in Celsius

and heart beat per minute(bpm). The temperature, heartbeat and location of soldier is sent to the defense system through SMS. In this way live location, health of soldier is monitored. If the higher or lower temperature,heart beat is detected the buzzer is automatically activated. In this project we are using Raspberry Pi for processing the input and output. All the sensors(DHT11,heartbeat sensor), GSM module and GPS module ,driver boards are connected to the Raspberry Pi. As we are using RaspberryPi we selected the embedded python as programming language. Generally embedded python consists several libraries which are useful for easy coding.

Schematic Pin Diagram:

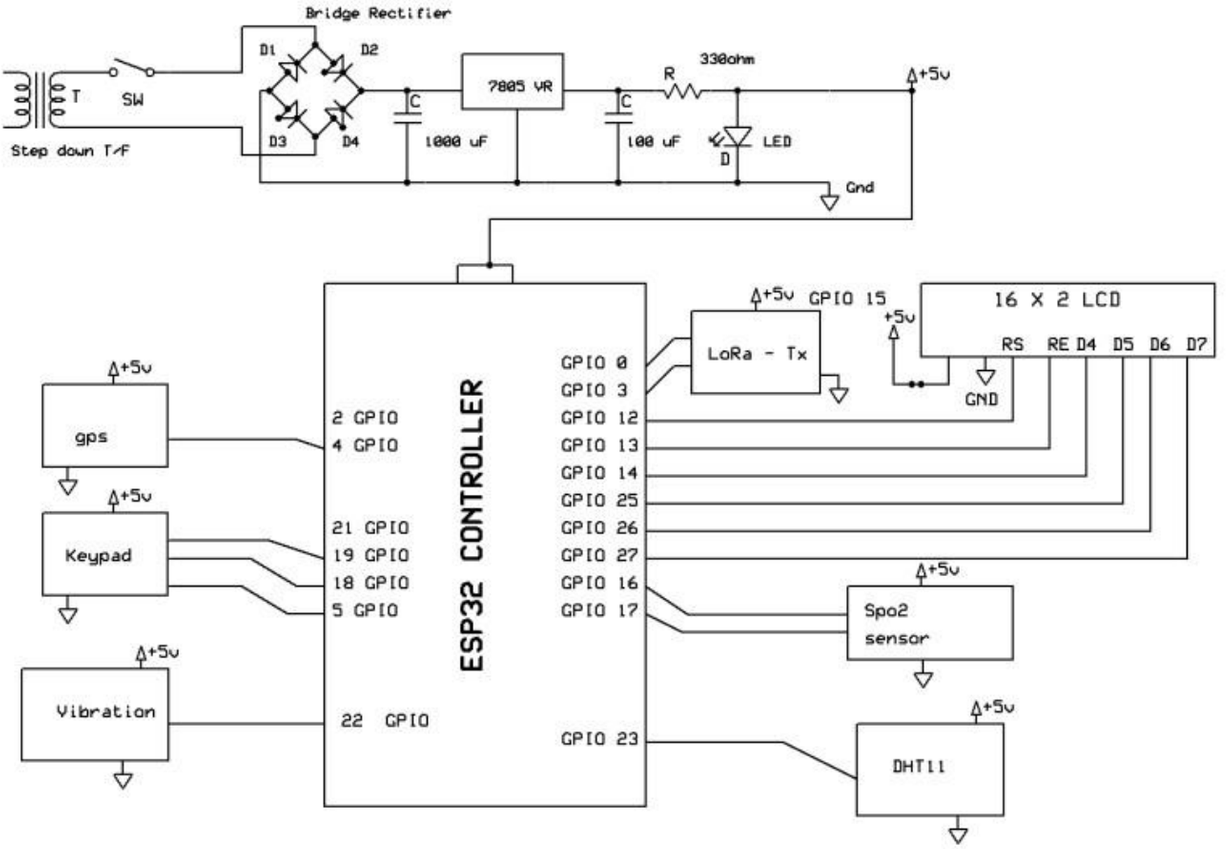


Fig 4.3: LORA (TX) Schematic Pin Diagram

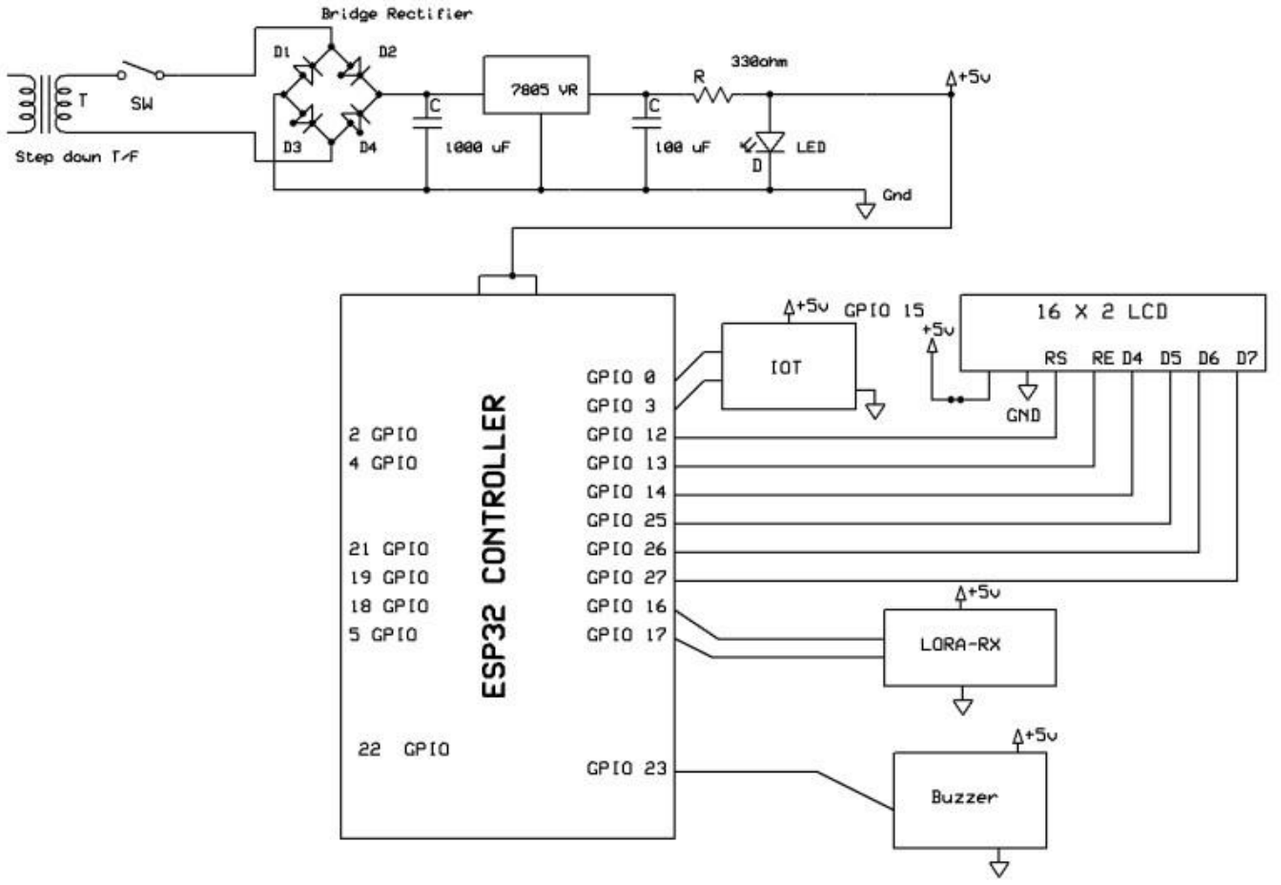


Fig 4.4: LORA (RX) Schematic diagram

The hardware components are connected to Arduino board. The General purpose I/O pins are connected to buzzer, LCD Display, GSM module, Temperature and humidity sensor, heart beat sensor and GPS modules. 5V RPS is connected to the raspberry pi. The RPS circuit diagram consists of stepdown transformer, bridge rectifier for AC to DC conversion. voltage regulator 7805 for supplying 5V voltage to the Arduino.

Hardware connected to the Raspberry pi:

- GSM module is connected to the 5V supply TX, RX of Arduino.
- D7, D6, D5, D4, RE, RS of LCD Display are connected to the D2 –D7.
- GPS module is connected to 8,9th pin.
- DHT11 sensor is connected to the 10th pin.
- Buzzer is connected to the 13th pin.
- Heartbeat sensor is connected to the A0.

CHAPTER – 5

EMBEDDED SYSTEMS

5.1 Embedded Systems:

An embedded system is a computer system designed to perform one or a few dedicated functions often with real-time computing constraints. It is embedded as part of a complete device often including hardware and mechanical parts. By contrast, a general-purpose computer, such as a personal computer (PC), is designed to be flexible and to meet a wide range of end-user needs. Embedded systems control many devices in common use today.

Embedded systems are controlled by one or more main processing cores that are typically either microcontrollers or digital signal processors (DSP). The key characteristic, however, is being dedicated to handle a particular task, which may require very powerful processors. For example, air traffic control systems may usefully be viewed as embedded, even though they involve mainframe computers and dedicated regional and national networks between airports and radar sites. (Each radar probably includes one or more embedded systems of its own.)

Since the embedded system is dedicated to specific tasks, design engineers can optimize it to reduce the size and cost of the product and increase the reliability and performance. Some embedded systems are mass-produced, benefiting from economies of scale.

Physically embedded systems range from portable devices such as digital watches and MP3 players to large stationary installations like traffic lights, factory controllers, or the systems controlling nuclear power plants. Complexity varies from low, with a single microcontroller chip, to very high with multiple units, peripherals and networks mounted inside a large chassis or enclosure.

In general, "embedded system" is not a strictly definable term, as most systems have some element of extensibility or programmability. For example, handheld computers share some elements with embedded systems such as the operating systems and microprocessors which power them, but they allow different applications to be loaded and peripherals to be connected. Moreover, even systems which don't expose programmability as a primary feature generally need to support software updates. On a continuum from "general purpose" to "embedded", large application systems will have subcomponents at most points even if the system as a whole is "designed to perform one or a few dedicated functions", and is thus appropriate to call "embedded". A modern example of embedded system is shown in fig: 2.1.

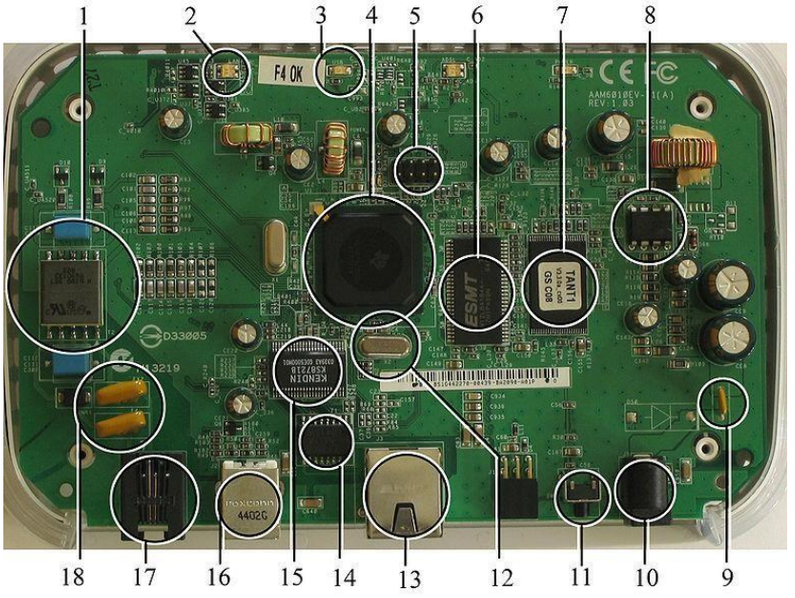


Fig 5.1: A modern example of embedded system

Labeled parts include microprocessor (4), RAM (6), flash memory (7). Embedded systems programming is not like normal PC programming. In many ways, programming for an embedded system is like programming PC 15 years ago. The hardware for the system is usually chosen to make the device as cheap as possible. Spending an extra dollar a unit in order to make things easier to program can cost millions. Hiring a programmer for an extra month is cheap in comparison. This means the programmer must make do with slow processors and low memory, while at the same time battling a need for efficiency not seen in most PC applications. Below is a list of issues specific to the embedded field.

5.1.1 History:

In the earliest years of computers in the 1930–40s, computers were sometimes dedicated to a single task but were far too large and expensive for most kinds of tasks performed by embedded computers of today. Over time however, the concept of programmable controllers evolved from traditional electromechanical sequencers, via solid state devices, to the use of computer technology.

One of the first recognizably modern embedded systems was the Apollo Guidance Computer, developed by Charles Stark Draper at the MIT Instrumentation Laboratory. At the project's inception, the Apollo guidance computer was considered the riskiest item in the Apollo project as it employed the then newly developed monolithic integrated circuits to reduce the size and weight. An early mass-produced embedded system was the Autonetics D-17 guidance computer for the Minuteman missile, released in 1961. It was built from transistor logic and had a hard disk for main memory. When the Minuteman II went into production in 1966, the D-17 was replaced with a new computer that was the first high-volume use of integrated circuits.

2.1.2 Tools:

Embedded development makes up a small fraction of total programming. There's also a large number of embedded architectures, unlike the PC world where 1 instruction set rules, and the Unix world where there's only 3 or 4 major ones. This means that the tools are more expensive. It also means that they're lowering featured, and less developed. On a major embedded project, at some point you will almost always find a compiler bug of some sort.

Debugging tools are another issue. Since you can't always run general programs on your embedded processor, you can't always run a debugger on it. This makes fixing your program difficult. Special hardware such as JTAG ports can overcome this issue in part. However, if you stop on a breakpoint when your system is controlling real world hardware (such as a motor), permanent equipment damage can occur. As a result, people doing embedded programming quickly become masters at using serial IO channels and error message style debugging.

5.1.3 Resources:

To save costs, embedded systems frequently have the cheapest processors that can do the job. This means your programs need to be written as efficiently as possible. When dealing with large data sets, issues like memory cache misses that never matter in PC programming can hurt you. Luckily, this won't happen too often- use reasonably efficient algorithms to start and optimize only when necessary. Of course, normal profilers won't work well, due to the same reason debuggers don't work well.

Memory is also an issue. For the same cost savings reasons, embedded systems usually have the least memory they can get away with. That means their algorithms must be memory efficient (unlike in PC programs, you will frequently sacrifice processor time for memory, rather than the reverse). It also means you can't afford to leak memory. Embedded applications generally use deterministic memory techniques and avoid the default "new" and "malloc" functions, so that leaks can be found and eliminated more easily. Other resources programmers expect may not even exist. For example, most embedded processors do not have hardware FPUs (Floating-Point Processing Unit). These resources either need to be emulated in software or avoided altogether.

5.1.4 Real Time Issues:

Embedded systems frequently control hardware and must be able to respond to them in real time. Failure to do so could cause inaccuracy in measurements, or even damage hardware such as motors. This is made even more difficult by the lack of resources available. Almost all embedded systems need to be able to prioritize some tasks over others, and to be able to put off/skip low priority tasks such as UI in favor of high priority tasks like hardware control.

5.2 Need for Embedded Systems:

The uses of embedded systems are virtually limitless, because every day new products are introduced to the market that utilizes embedded computers in novel ways. In recent years, hardware such as microprocessors, microcontrollers, and FPGA chips has become much cheaper. So, when implementing a new form of control, it's wiser to just buy the generic chip and write your own custom software for it. Producing a custom-made chip to handle a particular task or set of tasks costs far more time and money. Many embedded computers even come with extensive libraries, so that "writing your own software" becomes a very trivial task indeed. From an implementation viewpoint, there is a major difference between a computer and an embedded system. Embedded systems are often required to provide Real-Time response.

The main elements that make embedded systems unique are its reliability and ease in debugging.

5.2.1 Debugging:

Embedded debugging may be performed at different levels, depending on the facilities available. From simplest to most sophisticate, they can be roughly grouped into the following areas:

- Interactive resident debugging, using the simple shell provided by the embedded operating system (e.g. Forth and Basic)
- External debugging using logging or serial port output to trace operation using either a monitor in flash or using a debug server like the Remedy Debugger which even works for heterogeneous multi core systems.
- An in-circuit debugger (ICD), a hardware device that connects to the microprocessor via a JTAG or Nexus interface. This allows the operation of the microprocessor to be controlled externally, but is typically restricted to specific debugging capabilities in the processor.
- An in-circuit emulator replaces the microprocessor with a simulated equivalent, providing full control over all aspects of the microprocessor.
- A complete emulator provides a simulation of all aspects of the hardware, allowing all of it to be controlled and modified and allowing debugging on a normal PC.
- Unless restricted to external debugging, the programmer can typically load and run software through the tools, view the code running in the processor, and start or stop its operation. The view of the code may be as assembly code or source-code.

Because an embedded system is often composed of a wide variety of elements, the debugging strategy may vary. For instance, debugging a software (and microprocessor) centric embedded system is different from debugging an embedded system where most of the processing is performed by peripherals (DSP, FPGA, co-processor). An increasing number of embedded systems today use more than one single processor core. A common problem with multi-core development is the proper synchronization of software execution. In such a

case, the embedded system design may wish to check the data traffic on the busses between the processor cores, which requires very low-level debugging, at signal/bus level, with a logic analyzer, for instance.

5.2.2 Reliability:

Embedded systems often reside in machines that are expected to run continuously for years without errors and in some cases recover by them if an error occurs. Therefore, the software is usually developed and tested more carefully than that for personal computers, and unreliable mechanical moving parts such as disk drives, switches or buttons are avoided.

Specific reliability issues may include:

- The system cannot safely be shut down for repair, or it is too inaccessible to repair. Examples include space systems, undersea cables, navigational beacons, bore-hole systems, and automobiles.
- The system must be kept running for safety reasons. "Limp modes" are less tolerable. Often backups are selected by an operator. Examples include aircraft navigation, reactor control systems, safety critical chemical factory controls, train signals, engines on single-engine aircraft.
- The system will lose large amounts of money when shut down: Telephone switches, factory controls, bridge and elevator controls, funds transfer and market making, automated sales and service. A variety of techniques are used, sometimes in combination, to recover from errors—both software bugs such as memory leaks, and also soft errors in the hardware:
- Watchdog timer that resets the computer unless the software periodically notifies the watchdog
- Subsystems with redundant spares that can be switched over to
- software "limp modes" that provide partial function
- Designing with a Trusted Computing Base (TCB) architecture [6] ensures a highly secure & reliable system environment
- An Embedded Hypervisor is able to provide secure encapsulation for any subsystem component, so that a compromised software component cannot interfere with other subsystems, or privileged-level system software. This encapsulation keeps faults from propagating from one subsystem to another, improving reliability. This may also allow a subsystem to be automatically shut down and restarted on fault detection.
- Immunity Aware Programming **5.3 Explanation of Embedded Systems:**

5.3.1 Software Architecture:

There are several different types of software architecture in common use.

- Simple Control Loop:

In this design, the software simply has a loop. The loop calls subroutines, each of which manages a part of the hardware or software.

- Interrupt Controlled System:

Some embedded systems are predominantly interrupting controlled. This means that tasks performed by the system are triggered by different kinds of events. An interrupt could be generated for example by a timer in a predefined frequency, or by a serial port controller receiving a byte. These kinds of systems are used if event handlers need low latency and the event handlers are short and simple.

Usually, these kinds of systems run a simple task in a main loop also, but this task is not very sensitive to unexpected delays. Sometimes the interrupt handler will add longer tasks to a queue structure. Later, after the interrupt handler has finished, these tasks are executed by the main loop. This method brings the system close to a multitasking kernel with discrete processes.

- Cooperative Multitasking:

  A non-pre-emptive multitasking system is very similar to the simple control loop scheme, except that the loop is hidden in an API. The programmer defines a series of tasks, and each task gets its own environment to “run” in. When a task is idle, it calls an idle routine, usually called “pause”, “wait”, “yield”, “nop” (stands for no operation), etc. The advantages and disadvantages are very similar to the control loop, except that adding new software is easier, by simply writing a new task, or adding to the queue-interpreter.

- Primitive Multitasking:

In this type of system, a low-level piece of code switches between tasks or threads based on a timer (connected to an interrupt). This is the level at which the system is generally considered to have an "operating system" kernel. Depending on how much functionality is required, it introduces more or less of the complexities of managing multiple tasks running conceptually in parallel.

As any code can potentially damage the data of another task (except in larger systems using an MMU) programs must be carefully designed and tested, and access to shared data must be controlled by some synchronization strategy, such as message queues, semaphores or a non-blocking synchronization scheme.

Because of these complexities, it is common for organizations to buy a real-time operating system, allowing the application programmers to concentrate on device functionality rather than operating system services, at least for large systems; smaller systems often cannot afford the overhead associated with a generic real time system, due to limitations regarding memory size, performance, and/or battery life.

- Microkernels And Exokernels:

  A microkernel is a logical step up from a real-time OS. The usual arrangement is that the operating system kernel allocates memory and switches the CPU to different threads of execution. User mode processes implement major functions such as file systems, network interfaces, etc.

In general, microkernels succeed when the task switching and intertask communication is fast, and fail when they are slow. Exokernels communicate efficiently by normal subroutine calls. The hardware and all the software in the system are available to, and extensible by application programmers. Based on performance, functionality, requirement the embedded systems are divided into three categories:

5.3.2 Stand Alone Embedded System:

These systems take the input in the form of electrical signals from transducers or commands from human beings such as pressing of a button etc.., process them and produces desired output. This entire process of taking input, processing it and giving output is done in standalone mode. Such embedded systems comes under standalone embedded systems Eg: microwave oven, air conditioner etc.

5.3.3 Real-time embedded systems:

Embedded systems which are used to perform a specific task or operation in a specific time period those systems are called as real-time embedded systems. There are two types of real-time embedded systems.

- Hard Real-time embedded systems:

These embedded systems follow an absolute deadline time period i.e.., if the tasking is not done in a particular time period, then there is a cause of damage to the entire equipment.

Eg: consider a system in which we must open a valve within 30 milliseconds. If this valve is not opened in 30 ms this may cause damage to the entire equipment. So, in such cases we use embedded systems for doing automatic operations.

- Soft Real Time embedded systems:

These embedded systems follow a relative deadline time period i.e.., if the task is not done in a particular time that will not cause damage to the equipment.

Eg: Consider a TV remote control system, if the remote control takes a few milliseconds delay it will not cause damage either to the TV or to the remote control. These systems which will not cause damage when they are not operated at considerable time period those systems come under soft real-time embedded systems.

5.3.4 Network communication embedded systems:

A wide range network interfacing communication is provided by using embedded systems.

Eg:

- Consider a web camera that is connected to the computer with internet can be used to spread communication like sending pictures, images, videos etc.., to another computer with internet connection throughout anywhere in the world.
- Consider a web camera that is connected at the door lock.

Whenever a person comes near the door, it captures the image of a person and sends to the desktop of your computer which is connected to internet. This gives an alerting message with image on to the desktop of your

computer, and then you can open the door lock just by clicking the mouse. Fig: 5.2 show the network communications in embedded systems.

Fig 5.2: Network communication embedded systems

5.3.5 Different types of processing units:

The central processing unit (C.P.U.) can be any one of the following microprocessors, microcontroller, digital signal processing.

- Among these Microcontroller is of low-cost processor and one of the main advantage of microcontrollers is, the components such as memory, serial communication interfaces, analog to digital converters etc.., all these are built on a single chip. The numbers of external components that are connected to it are very less according to the application.
- Microprocessors are more powerful than microcontrollers. They are used in major applications with a number of tasking requirements. But the microprocessor requires many external components like memory, serial communication, hard disk, input output ports etc.., so the power consumption is also very high when compared to microcontrollers.
- Digital signal processing is used mainly for the applications that particularly involved with processing of signals

5.4 APPLICATIONS OF EMBEDDED SYSTEMS:

5.4.1 Consumer applications:

At home we use a number of embedded systems which include microwave oven, remote control, vcd players, dvd players, camera etc….

Fig5.3: Automatic coffee makes equipment

5.4.2 Office automation:

We use systems like fax machine, modem, printer etc…

Fig5.4: Fax machine

Fig5.5: Printing machine

5.4.3. Industrial automation:

Today a lot of industries are using embedded systems for process control. In industries we design the embedded systems to perform a specific operation like monitoring temperature, pressure, humidity ,voltage, current etc.., and basing on these monitored levels we do control other devices, we can send information to a centralized monitoring station.

Fig5.6: Robot

In critical industries where human presence is avoided there, we can use robot**s** which are programmed to do a specific operation.

5.4.5 Computer networking:

Embedded systems are used as bridges routers etc.

Fig5.7: Computer networking

5.4.6 Tele communications:

Cell phones, web cameras etc.

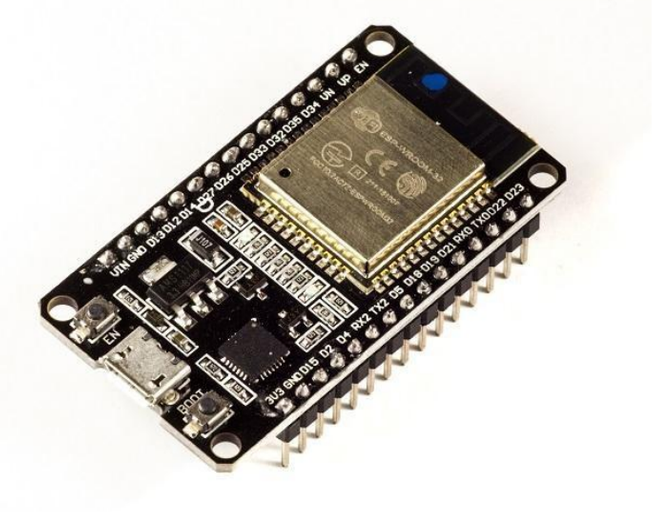
Fig5.8: Cell Phone

Fig5.9: Web camera

CHAPTER – 6
HARDWARE DESCRIPTION

6.1: ESP32 CONTROLLER

ESP32 is the SoC (System on Chip) microcontroller which has gained massive popularity recently. Whether the popularity of ESP32 grew because of the growth of IoT or whether IoT grew because of the introduction of ESP32 is debatable. If you know 10 people who have been part of the firmware



6.1: ESP 32 controller

development for any IoT device, chances are that 7−8 of them would have worked on ESP32 at some point.

Before we delve into the actual reasons for the popularity of ESP32, let's take a look at some of its important specifications. The specs listed below belong to the ESP32 WROOM 32 variant

- Integrated Crystal− 40 MHz
- Module Interfaces− UART, SPI, I2C, PWM, ADC, DAC, GPIO, pulse counter, capacitive touch sensor
- Integrated SPI flash− 4 MB
- ROM− 448 KB (for booting and core functions)
- SRAM− 520 KB
- Integrated Connectivity Protocols− Wi-Fi, Bluetooth, BLE
- On−chip sensor− Hall sensor
- Operating temperature range− −40 − 85 degrees Celsius
- Operating Voltage− 3.3V

- Operating Current− 80 mA (average)

With the above specifications in front of you, it is very easy to decipher the reasons for ESP32's popularity. Consider the requirements an IoT device would have from its microcontroller (μC). If you've gone through the previous chapter, you'd have realized that the major operational blocks of any IoT device are sensing, processing, storage, and transmitting. Therefore, to begin with, the μC should be able to interface with a variety of sensors. It should support all the common communication protocols required for sensor interface: UART, I2C, SPI. It should have ADC and pulse counting capabilities. ESP32 fulfills all of these requirements. On top of that, it also can interface with capacitive touch sensors. Therefore, most common sensors can interface seamlessly with ESP32.

Secondly, the μC should be able to perform basic processing of the incoming sensor data, sometimes at high speeds, and have sufficient memory to store the data. ESP32 has a max operating frequency of 40 MHz, which is sufficiently high. It has two cores, allowing parallel processing, which is a further add-on. Finally, its 520 KB SRAM is sufficiently large for processing a large array of data onboard. Many popular processes and transforms, like FFT, peak detection, RMS calculation, etc. can be performed onboard ESP32. On the storage front, ESP32 goes a step ahead of the conventional microcontrollers and provides a file system within the flash. Out of the 4 MB of onboard flash, by default, 1.5 MB is reserved as SPIFFS (SPI Flash File System). Think of it as a mini−SD Card that lies within the chip itself. You can not only store data, but also text files, images, HTML and CSS files, and a lot more within SPIFFS. People have displayed beautiful WebPages on WiFi servers created using ESP32, by storing HTML files within SPIFFS.

Finally, for transmitting data, ESP32 has integrated WiFi and Bluetooth stacks, which have proven to be a game-changer. No need to connect a separate module (like a GSM module or an LTE module) for testing cloud communication. Just have the ESP32 board and a running WiFi, and you can get started. ESP32 allows you to use WiFi in Access Point as well as Station Mode. While it supports TCP/IP, HTTP, MQTT, and other traditional communication protocols, it also supports HTTPS. Yep, you heard that right. It has a crypto−core or a crypto-accelerator, a dedicated piece of hardware whose job is to accelerate the encryption process. So you cannot only communicate with your web server, you can do so securely. BLE support is also critical for several applications. Of course, you can interface LTE or GSM or LoRa modules with ESP32. Therefore, on the 'transmitting data' front as well, ESP32 exceeds expectations.

ESP32 is a chip that provides Wi-Fi and (in some models) Bluetooth connectivity for embedded devices – in other words, for IoT devices. While ESP32 is technically just the chip, the modules and development boards that contain this chip are often also referred to as “ESP32” by the manufacturer.

The original ESP32 chip had a single core Tensilica Xtensa LX6 microprocessor. The processor had a clock rate of over 240 MHz, which made for a relatively high data processing speed.

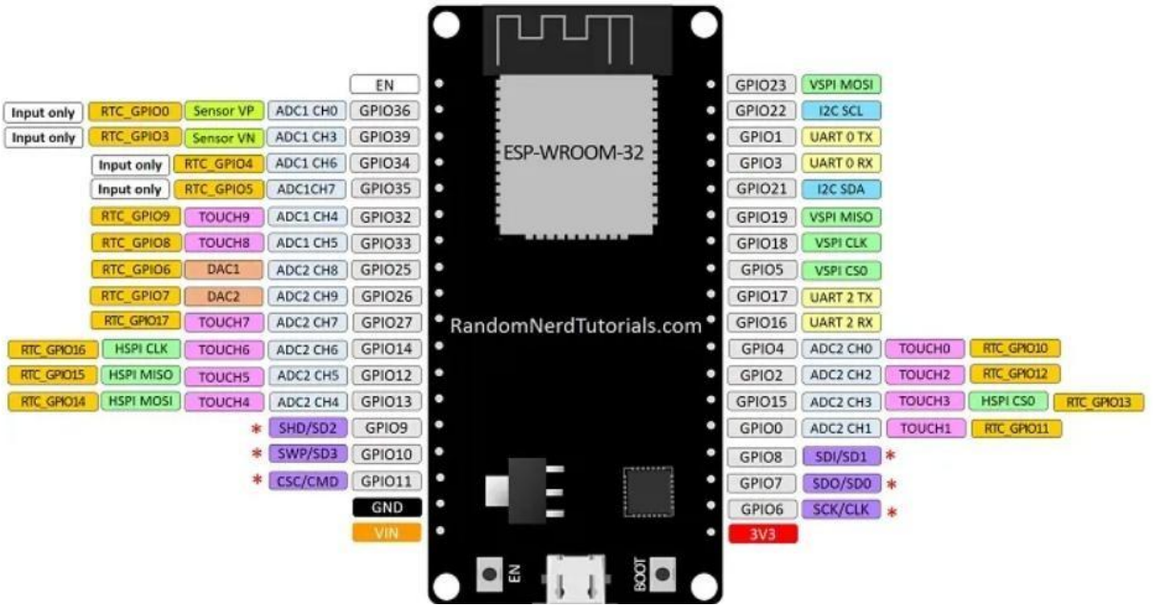
More recently, new models were added, including the ESP32-C and -S series, which include both single and dual core variations. These two series also rely on a Risc-V CPU model instead of Xtensa. Risc-V is similar to the ARM architecture, which is well-supported and well-known, but Risc-V is open source and easy to use. Specifically, Risc-V and ARM have good support from GNU compilers, while the Xtensa needed extra support and development to work with the compilers.

The newer models are available with combined Wi-Fi and Bluetooth connectivity, or just Wi-Fi connectivity. There are several different chip models available, including:

- ESP32-D0WDQ6 (and ESP32D0WD)
- ESP32-D2WD
- ESP32-S0WD
- System in package (SiP) – ESP32-PICO-D4
- ESP32 S series
- ESP32-C series
- ESP32-H series

The ESP32 is most commonly used for mobile devices, wearable tech, and IoT applications, such as Nabto Edge. Moreover, since Mongoose OS introduced an ESP32 IoT Starter Kit, the ESP32 has gained a reputation as the ultimate chip for hobbyists and IoT developers. It’s suitable for commercial IoT, and its capabilities and resources have grown impressively over the past four years.

ESP32 features and specifications:



6.2: ESP32 Pin diagram

ESP-32	DESCRIPTION
Core	2
Architecture	32 bits
Clock	Tensilica Xtensa LX106 160-240MHz
WiFi	IEEE802.11 b/g/n
Bluetooth	Yes - classic & BLE
RAM	520KB
Flash	External QSPI - 16MB
GPIO	22
DAC	2
ADC	18
Interfaces	SPI-I2C-UART-I2S-CAN

6.1: Here's a high-level summary of the features and specifications of the ESP32:

- **Processors** – The ESP32 uses a Tensilica Xtensa 32-bit LX6 microprocessor. This typically relies on a dual core architecture, with the exception of one module, the ESP32-S0WD, which uses a singlecore system. The clock frequency reaches up to 240MHz and it performs up to 600 DMIPS (Dhrystone

millions of instructions per second). Moreover, its low power consumption allows for analog to digital conversions as well as computation and level thresholds, even while the chip is in deep sleep mode.

- **Wireless connectivity** – The ESP32 enables connectivity to integrated Wi-Fi through the 802.11 b/g/n/e/i/. Moreover, Bluetooth connectivity is made possible with the v4.2 BR/EDR, and the series also features Bluetooth low energy (BLE).

- **Memory** – Internal memory for the ESP32 is as follows. ROM: 448 KB (for booting/core functions), SRAM: 520 KB (for data/instructions), RTC fast SRAM: 8 KB (for data storage/main CPU during boot from sleep mode), RTC slow SRAM: 8 KB (for co-processor access during sleep mode), and eFuse: 1 KiBit (256 bits used for the system (MAC address and chip configuration) and 768 bits reserved for customer applications). Moreover, some of the ESP32 chips, including the ESP32D2WD and ESP32-PICO-D4, have internally connected flash. See the respective internal flash memory for each chip in the ESP32 Chips section.

- **External flash and SRAM** – ESP32 supports up to four 16 MB external QSPI flashes and SRAMs with hardware encryption based on AES to protect developers' programs and data. It accesses the external QSPI flash and SRAM through high-speed caches.

- **Security** – The ESP32 supports all IEEE 802.11 standard security features, including WFA, WPA/WPA2 and WAPI. Moreover, ESP32 has a secure boot and flash encryption.

ESP32 functions:

ESP32 has many applications when it comes to IoT. Here are just some of the IoT functions the chip is used for**:**

- **Networking**: The module's Wi-Fi antenna and dual core enables embedded devices to connect to routers and transmit data.

- **Data processing**: Includes processing basic inputs from analog and digital sensors to far more complex calculations with an RTOS or non-OS software development kit (SDK). A non-OS SDK refers to one that is designed to run directly on the chip without a full operating system supporting it.

- **P2P connectivity**: Creates direct communication between different ESPs and other devices using IoT P2P connectivity.

- **Web server**: Provides access to pages written in HTML or development languages.

ESP32 Applications:

The ESP32 modules are commonly found in the following IoT devices:

- Smart industrial devices, including programmable logic controllers (PLCs)
- Smart medical devices, including wearable health monitors
- Smart energy devices, including HVAC and thermostats
- Smart security devices, including surveillance cameras and smart locks

Chip versus modules versus development boards

The ESP32 is just the name of the chip. Device manufacturers and developers have three different format choices, and the decision of which one to go with will depend on their individual circumstances:

- **ESP32 chip**: This is the bare-bones chip that is manufactured by Espressif. It comes unshielded, meaning there's no protective casing, and it can't be attached to a module or board without soldering. Therefore, most device manufacturers do not purchase just the chip, as this will add an additional layer of complexity to the production process.
- **ESP32 modules**: These are surface mountable modules that contain the chip. The modules are essentially small electrical components that can be attached to a circuit board. The benefit here is that you can easily mount these modules onto an MCU board. The chip is also usually shielded and preapproved by the FCC, which means device manufacturers do not need to worry about adding additional steps to the production process to achieve FCC compliance regarding Wi-Fi shielding.
- **ESP32 development boards:** These are IoT MCU development boards that have the modules containing the ESP32 chip preinstalled. They are used by hobbyists, device manufacturers and developers to test and prototype IoT devices before entering mass production. There is a wide variety of makes and models of ESP32 development boards, produced by different manufacturers. Here are some important specs to consider when choosing a suitable IoT ESP32 development board:
- GPIO pins
- ADC pins
- Wi-Fi antennas
- LEDs
- Shielding*
- Flash memory

*Many international markets require shielded Wi-Fi devices, as Wi-Fi produces a lot of radio frequency interference (RFI), and shielding minimizes this interference. This should, therefore, be a key consideration for all developers and embedded device manufacturers.

6.2: Power Supply:

A regulated power supply converts unregulated AC (Alternating Current) to a constant DC (Direct Current). A regulated power supply is used to ensure that the output remains constant even if the input changes. A regulated DC power supply is also known as a linear power supply; it is an embedded circuit and consists of various blocks. The regulated power supply will accept an AC input and give a constant DC output. The figure below shows the block diagram of a typical regulated DC power supply.

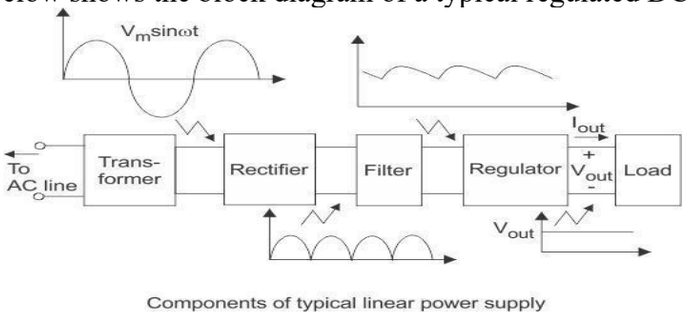


Figure 6.3: Block diagram of power supply

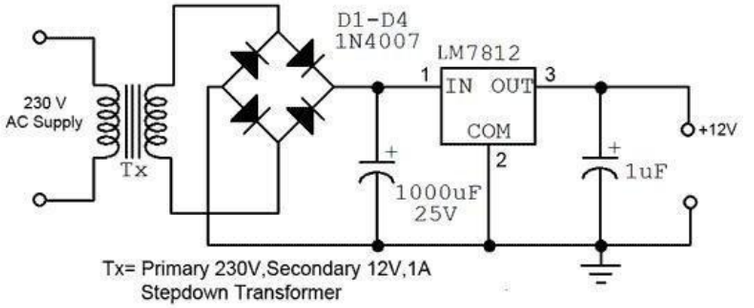
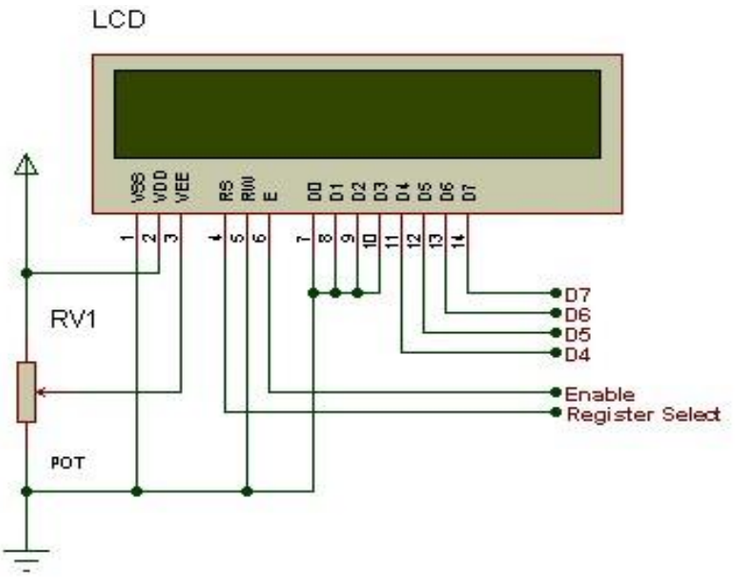


Figure 6.4: Power supply circuit diagram

6.3: LCD DISPLAY

LCD Background: One of the most common devices attached to a micro controller is an LCD display. Some of the most common LCD's connected to the many microcontrollers are 16x2 and 20x2 displays. This means 16 characters per line by 2 lines and 20 characters per line by 2 lines, respectively.



6.5: LCD Pin diagram

Table 6.2: Character LCD pins with Microcontroller

Pin No.	**Name**	**Description**
Pin no. 1	VSS	Power supply (GND)
Pin no. 2	VCC	Power supply (+5V)
Pin no. 3	VEE	Contrast adjustment
Pin no. 4	RS	0 = Instruction input, 1 = Data input
Pin no. 5	R/W	0 = Write to LCD module, 1 = Read from LCD module
Pin no. 6	EN	Enable signal
Pin no. 7	D0	Data bus line 0 (LSB)
Pin no. 8	D1	Data bus line 1
Pin no. 9	D2	Data bus line 2
Pin no. 10	D3	Data bus line 3
Pin no. 11	D4	Data bus line 4
Pin no. 12	D5	Data bus line 5
Pin no. 13	D6	Data bus line 6
Pin no. 14	D7	Data bus line 7 (MSB)

The LCD requires 3 control lines as well as either 4 or 8 I/O lines for the data bus. The user may select whether the LCD is to operate with a 4-bit data bus or an 8-bit data bus. If a 4-bit data bus is used the LCD will require a total of 7 data lines (3 control lines plus the 4 lines for the data bus). If an 8-bit data bus is used the LCD will require a total of 11 data lines (3 control lines plus the 8 lines for the data bus).

The three control lines are referred to as **EN**, **RS**, and **RW**.

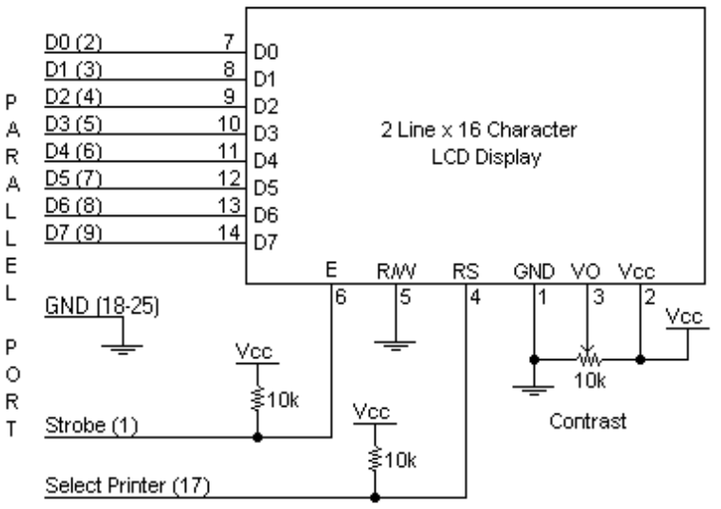
The **EN** line is called "Enable." This control line is used to tell the LCD that we are sending it data. To send data to the LCD, our program should make sure this line is low (0) and then set the other two control lines and/or put data on the data bus. When the other lines are completely ready, bring **EN** high (1) and wait for the minimum amount of time required by the LCD datasheet (this varies from LCD to LCD), and end by bringing it low (0) again.

The **RS** line is the "Register Select" line. When RS is low (0), the data is to be treated as a command or special instruction (such as clear screen, position cursor, etc.). When RS is high (1), the data being sent is text data which should be displayed on the screen. For example, to display the letter "T" on the screen we would set RS high.

The **RW** line is the "Read/Write" control line. When RW is low (0), the information on the data bus is being written to the LCD. When RW is high (1), the program is effectively querying (or reading) the LCD. Only one instruction ("Get LCD status") is a read command. All others are write commands--so RW will almost always be low.

Finally, the data bus consists of 4 or 8 lines (depending on the mode of operation selected by the user). In the case of an 8-bit data bus, the lines are referred to as DB0, DB1, DB2, DB3, DB4, DB5, DB6, and DB7.

Schematic:



6.6: circuit description

Circuit Description:

Above is the quite simple schematic. The LCD panel's Enable and Register Select is connected to the Control Port. The Control Port is an open collector / open drain output. While most Parallel Ports have internal pull-up resistors, there is a few which don't. Therefore, by incorporating the two 10K external pull up resistors, the circuit is more portable for a wider range of computers, some of which may have no internal pull up resistors.

We make no effort to place the Data bus into reverse direction. Therefore, we hard wire the R/W line of the LCD panel, into write mode. This will cause no bus conflicts on the data lines. As a result, we cannot read back the LCD's internal Busy Flag which tells us if the LCD has accepted and finished processing the last instruction. This problem is overcome by inserting known delays into our program.

The 10k Potentiometer controls the contrast of the LCD panel. Nothing fancy here. As with all the examples, I've left the power supply out. We can use a bench power supply set to 5v or use an onboard +5 regulator. Remember a few de-coupling capacitors, especially if we have trouble with the

6.4: GPS MODULE

The Global Positioning System (GPS) is a burgeoning technology, which provides unequalled accuracy and flexibility of positioning for navigation, surveying and GIS data capture. The GPS NAVSTAR (Navigation Satellite timing and Ranging Global Positioning System) is a satellite-based navigation, timing and positioning system. The GPS provides continuous three-dimensional positioning 24 hrs a day throughout the world. The technology seems to be beneficiary to the GPS user community in terms of obtaining accurate data up to about100 meters for navigation, meter-level for mapping, and down to millimeter level for geodetic positioning. The GPS technology has tremendous amount of applications in GIS data collection, surveying, and mapping.

6.7: GPS Module

The Global Positioning System (GPS) is a U.S. space-based radio navigation system that provides reliable positioning, navigation, and timing services to civilian users on a continuous worldwide basis -- freely available to all. For anyone with a GPS receiver, the system will provide location and time. GPS provides accurate location and time information for an unlimited number of people in all weather, day and night, anywhere in the world.

The Global Positioning System (GPS) is a satellite-based navigation system made up of a network of 24 satellites placed into orbit by the U.S. Department of Defense. GPS was originally intended for military applications, but in the 1980s, the government made the system available for civilian use. GPS works in any

weather conditions, anywhere in the world, 24 hours a day. There are no subscription fees or setup charges to use GPS.

The GPS is made up of three parts: satellites orbiting the Earth; control and monitoring stations on Earth; and the GPS receivers owned by users. GPS satellites broadcast signals from space that are picked up and identified by GPS receivers. Each GPS receiver then provides three-dimensional location (latitude, longitude, and altitude) plus the time.

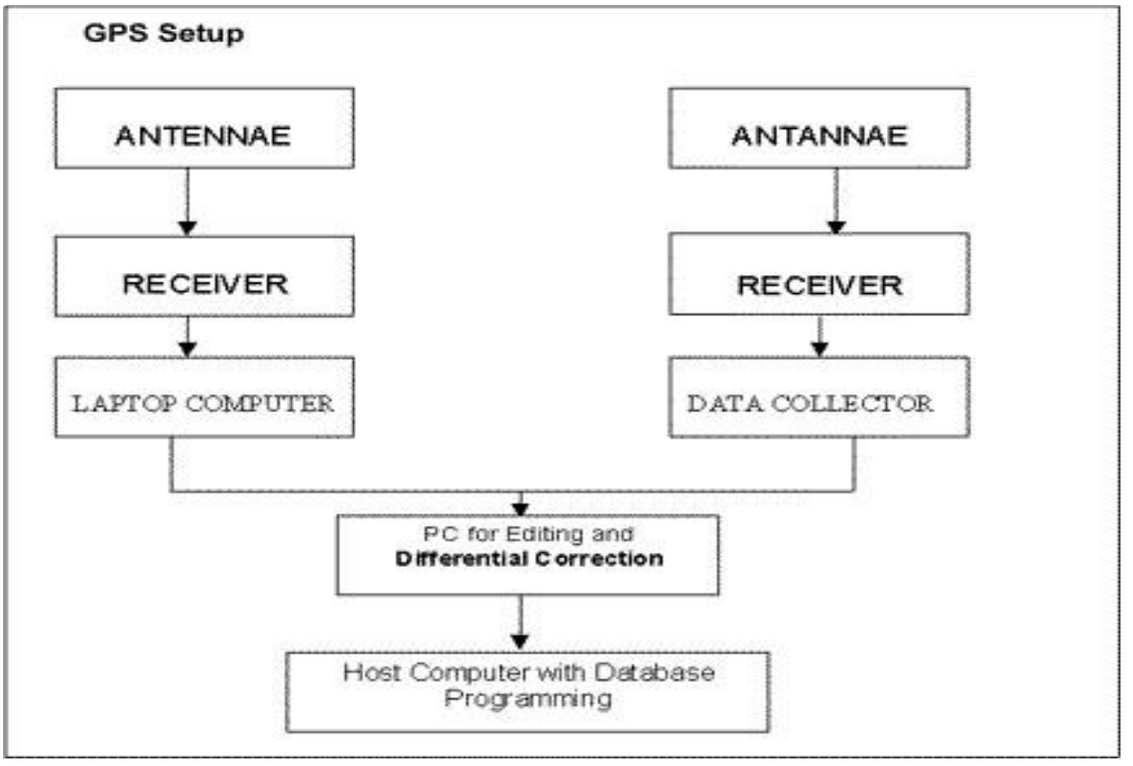
Individuals may purchase GPS handsets that are readily available through commercial retailers. Equipped with these GPS receivers, users can accurately locate where they are and easily navigate to where they want to go, whether walking, driving, flying, or boating. GPS has become a mainstay of transportation systems worldwide, providing navigation for aviation, ground, and maritime operations. Disaster relief and emergency services depend upon GPS for location and timing capabilities in their life-saving missions. Everyday activities such as banking, mobile phone operations, and even the control of power grids, are facilitated by the accurate timing provided by GPS. Farmers, surveyors, geologists and countless others perform their work more efficiently, safely, economically, and accurately using the free and open GPS signals.

Geo positioning -- Basic Concepts:

By positioning we understand the determination of stationary or moving objects. These can be determined as follows:

1. In relation to a well-defined coordinate system, usually by three coordinate values and
2. In relation to other point, taking one point as the origin of a local coordinate system.

The first mode of positioning is known as point positioning, the second as relative positioning. If the object to be positioned is stationary, we term it as static positioning. When the object is moving, we call it kinematics positioning. Usually, the static positioning is used in surveying and the kinematics position in navigation.



6.8: GPS Block Diagram

This is a complete GPS module that is based on the NEO-6M. This unit uses the latest technology to give the best possible positioning information and includes a larger built-in 25 x 25mm active GPS antenna with a UART TTL socket. A battery is also included so that you can obtain a GPS lock faster. This is an updated GPS module that can be used with ardupilot mega v2. This GPS module gives the best possible position information, allowing for better performance with your Ardupilot or other Multirotor control platform.

The NEO-6M GPS engine on this board is a quite good one, with the high precision binary output. It has also high sensitivity for indoor applications. NEO-6M GPS Module has a battery for power backup and EEPROM for storing configuration settings. The antenna is connected to the module through a ufl cable which allows for flexibility in mounting the GPS such that the antenna will always see the sky for best performance. This makes it powerful to use with cars and other mobile applications.

The GPS module has serial TTL output, it has four pins: TX, RX, VCC, and GND. You can download the ucenter software for configuring the GPS and changing the settings and much more.

Features NEO-6M GPS Module:-

- 5Hz position update rate
- Operating temperature range: -40 TO 85°CUART TTL socket
- EEPROM to save configuration settings
- Rechargeable battery for Backup
- The cold start time of 38 s and Hot start time of 1 s
- Supply voltage: 3.3 V
- Configurable from 4800 Baud to 115200 Baud rates. (default 9600)
- SuperSense ® Indoor GPS: -162 dBm tracking sensitivity

- Support SBAS (WAAS, EGNOS, MSAS, GAGAN)
- Separated 18X18mm GPS antenna

6.5: Heart Beat Sensor

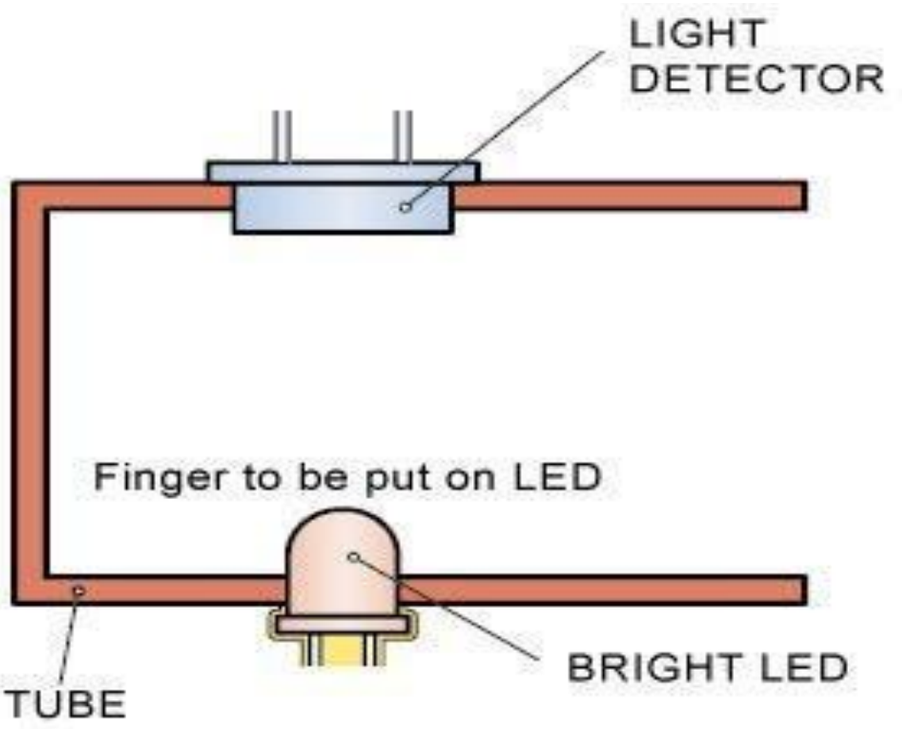
This heart beat sensor is designed to give digital output of heat beat when a finger is placed inside it. When the heart detector is working, the top-most LED flashes in unison with each heart beat. This digital output can be connected to microcontroller directly to measure the Beats Per Minute (BPM) rate. It works on the principle of light modulation by blood flow through finger at each pulse.

Features

- Heat beat indication by LED
- Instant output digital signal for directly connecting to microcontroller
- Applications
- Working Voltage +5V DC
- Digital Heart Rate monitor
- Bio-Feedback control of robotics and applications
- Exercise machines

Working

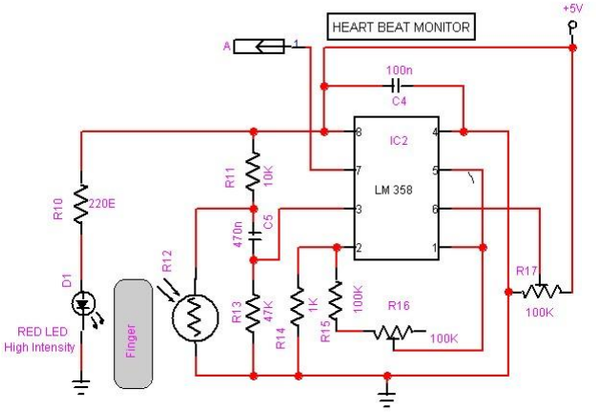
The sensor consists of a super bright red LED and light detector. The LED needs to be super bright as the light must pass through finger and detected at other end. Now, when the heart pumps a pulse of blood through the blood vessels, the finger becomes slightly more opaque and so less light reached the detector. With each heart pulse the detector signal varies. This variation is converted to electrical pulse. This signal is amplified and triggered through an amplifier which outputs +5V logic level signal. The output signal is also indicated on top by a LED which blinks on each heart beat. Following figure shows signal of heart beat and sensor signal output graph.



6.9: HEARTBEAT SENSOR

.

AMPLIFIER CIRCUIT:



6.10: Amplifier circuit

For amplification, we use IC LM 358. Pulse rate is sensed by using a high intensity type LED and LDR. The finger is inserted in probe and red light from high intensity LED is allowed to fall on the finger. The amount of red light absorbed by finger varies according to the pulsatile blood flow in the finger. Therefore the amount of light transmitted varies according to the blood flow.

The LDR placed on opposite side of LED detects the transmitted light. With increase in transmitted light its resistance decreases and vice-versa. A voltage divider circuit is employed to get a voltage signal proportional to the resistance of the LDR.

This voltage signal consists of AC and DC components. Non-moving structures (veins, blood capillaries, bones, soft tissues, non-pulsatile blood) absorb constant amount of light and hence contribute to the DC component of voltage signal. As it provides no information about the blood pulses, DC components are not needed. Pulsatile blood absorbs varying amount of light and hence contributes to AC component of voltage signal. AC components are our required signal.

The magnitude of the DC components is almost 100-1000 times higher than the AC components. Hence they need to be removed in order for the AC components to be conditioned properly further on. Therefore, a high pass filter circuit is employed after the voltage divider network to block the DC components of the signal.

6.6: DHT11- Humidity and Temperature Sensor

DHT11 digital temperature and humidity sensor is a calibrated digital signal output of the temperature and humidity combined sensor. It uses a dedicated digital modules capture technology and the temperature and humidity sensor technology to ensure that products with high reliability and excellent long-term stability. Sensor includes a resistive element and a sense of wet NTC temperature measurement devices, and with a high-performance 8-bit microcontroller connected .

Humidity Sensor is one of the most important devices that has been widely in consumer, industrial, biomedical, and environmental etc. applications for measuring and monitoring Humidity.Humidity is defined as the amount of water present in the surrounding air. This water content in the air is a key factor in the wellness of mankind. For example, we will feel comfortable even if the temperature is 00C with less humidity i.e. the air is dry.

But if the temperature is 100C and the humidity is high i.e. the water content of air is high, then we will feel quite uncomfortable. Humidity is also a major factor for operating sensitive equipment like electronics, industrial equipment, electrostatic sensitive devices and high voltage devices etc. Such sensitive equipment must be operated in a humidity environment that is suitable for the device.

6.11: DHT 11

Hence, sensing, measuring, monitoring and controlling humidity is a very important task. Some of the important areas of application for sensing, measuring and controlling Humidity are mentioned below.

Domestic: Sensing and controlling humidity in our homes and offices is important as higher humidity conditions will affect the blood flow. Other areas include cooking, indoor plantation etc.

Industrial: In industries like refineries, chemical, metal, or other industries where furnaces are used, high humidity will reduce the amount of oxygen in the air and hence reduces the firing rate. Other industries like food processing, textile, paper etc. also need control of humidity.

Moisture: Generally, the term Moisture means water content of any material or substance. But practically, the term Moisture refers to the water content in solids and liquids. The term Humidity refers to the water content in gases (air).

Absolute Humidity: Absolute Humidity (AH) is the ratio of mass of the water vapour to the volume of the air. If m is the mass of the water vapour and V is the total volume i.e. volume of air and water vapour mixture, then Absolute Humidity AH is given by

$$\mathbf{AH = m/V}$$

Absolute Humidity doesn't take temperature into account, but it changes with temperature and pressure.

Relative Humidity: Whenever we talk about measuring Humidity, it usually Relative Humidity that we are talking about (unless otherwise specified).

Relative Humidity or RH is the ratio of the actual water vapour pressure present in the air at a temperature to the maximum water vapour pressure present in the air at the same temperature.

In weather reports and forecasts, the probability of precipitation or dew or fog is indicated using Relative Humidity and hence, it is considered an important metric.

Relative Humidity takes both temperature and pressure in to consideration. Hence, the Humidity Sensors which measure Relative Humidity, measure both the moisture content as well as the air temperature.

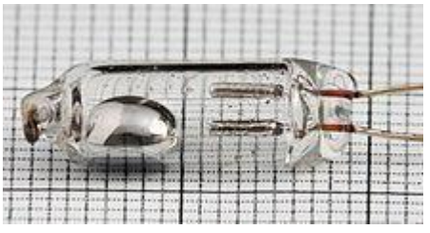
NOTE: For temperatures above 1000C, measuring Relative Humidity (RH) is of no use as it would deliver misleading values.

Specific Humidity: Specific Humidity (SH) is the ratio of mass of the water vapor to the total mass of the air. Mixing Ratio or Humidity Ratio: Mixing Ratio is the ratio of mass of the water vapor to mass of the dry air.

Dew Point Temperature: Dew Point Temperature is the temperature at with the water vapor content is saturated in the air. At Dew Point temperature, the Relative Humidity RH is 100%. In other words, for the air to hold maximum amount of water vapor (or moisture), it has to reach Dew Point Temperature.

6.7 **Mercury switch**

A mercury switch is an electrical switch that opens and closes a circuit when a small amount of the liquid metal mercury connects metal electrodes to close the circuit. There are several different basic designs (tilt, displacement, radial, etc.) but they all share the common design strength of non-eroding switch contacts.



6.12: MERCURY SWITCH

The most common is the mercury tilt switch. It is in one state (open or closed) when tilted one direction with respect to horizontal, and the other state when tilted the other direction. This is what older style thermostats used to turn a heater or air conditioner on or off.

The mercury displacement switch uses a 'plunger' that dips into a pool of mercury, raising the level in the container to contact at least one electrode. This design is used in relays in industrial applications that need to switch high current loads frequently. These relays use electromagnetic coils to pull steel sleeves inside hermetically sealed containers.

Description:

Mercury switches have one or more sets of electrical contacts in a sealed glass envelope that contains a small quantity of mercury. The envelope may also contain air, an inert gas, or a vacuum. Gravity constantly pulls the drop of mercury to the lowest point in the envelope. When the switch is tilted in the appropriate direction, the mercury touches a set of contacts, thus completing an electrical circuit. Tilting the switch in the opposite direction moves the mercury away from that set of contacts, breaking that circuit.[1] The switch may contain multiple sets of contacts, closing different sets at different angles, allowing, for example, single-pole, doublethrow (*SPDT*) operation.

Advantages

Mercury switches offer several advantages over other switch types:

- The contacts are enclosed, so oxidation of the contact points is unlikely.
- In hazardous locations, interrupting the circuit does not emit a spark that could ignite flammable gases.
- Contacts stay clean, and even if an internal arc occurs, the contact surfaces renew on every operation, so they don't wear out.
- Even a small drop of mercury has low resistance, so switches can carry useful amounts of current in a small size.[2]
- Sensitivity of the drop to gravity provides a unique sensing function, and lends itself to simple, low-force mechanisms for manual or automatic operation.
- The switches are quiet, as no contacts abruptly snap together.
- The mass of the moving mercury drop provides an over center effect to avoid chattering as the switch tilts.
- The envelope can include contacts for two or more circuits.

3.8: LORA

LoRa (**Lo**ng **Ra**nge) is a low-power wide-area network (**LPWAN)** technology. It is based on spread spectrum modulation techniques derived from chirp spread spectrum (CSS) technology.
LoRa uses license-free sub-gigahertz radio frequencybands like 433 MHz, 868 MHz (Europe), 915 MHz (Australia and North America) and 923 MHz (Asia). LoRa enables long-range transmissions (more than 10 km in rural areas) with low power consumption. The technology covers the physical layer, while other technologies and protocols such as LoRaWAN (Long Range Wide Area Network) cover the upper layers.

In a **LoRa (Long Range) module**, the terms **TX (transmit)** and **RX (receive)** refer to the module's ability to send and receive data over a wireless link. Here's a detailed explanation of the **TX** and **RX** functionality in the context of a LoRa module:

1. TX (Transmit) Mode:

When the LoRa module operates in TX mode, it is transmitting data to another device or module. The process of transmission involves the following:

- **Transmission of Packets**: In TX mode, the module sends a packet of data, which could include sensor readings, control commands, or other information, over long distances using the LoRa modulation technique.
- **LoRa Modulation**: LoRa uses a chirp spread spectrum (CSS) modulation, which spreads the signal over a wide bandwidth, ensuring the transmission is robust against interference and noise, and allows for long-distance communication at lower data rates.
- **Transmission Power**: The transmission power can be configured to control the range of communication. LoRa modules allow adjustments of output power (often up to +20 dBm or more) to strike a balance between communication range and power consumption.
- **Low Power TX Mode**: Some LoRa modules have low-power transmission modes, where they send data intermittently to save power. This is particularly useful in battery-powered IoT devices that need to transmit data over long periods.
- **Adjustable Parameters**: Parameters such as Spreading Factor (SF), Bandwidth (BW), and Coding Rate (CR) can be tuned to adjust the range, data rate, and robustness of transmission. A higher spreading factor results in a longer range but lower data rate.

6.13: LORA Module

LoRa uses a proprietary spread spectrum modulation that is similar to and a derivative of Chirp spread spectrum (CSS) modulation. The spread spectrum LoRa modulation is performed by representing each bit of payload information by multiple chirps of information. The rate at which the spread information is sent is referred to as the symbol rate, the ratio between the nominal symbol rate and chirp rate is the spreading factor (SF) and represents the number of symbols sent per bit of information.[1] LoRa can trade off data rate for sensitivity with a fixed channel bandwidth by selecting the amount of spread used (a selectable radio parameter

from 7 to 12). Lower SF means more chirps are sent per second; hence, you can encode more data per second. Higher SF implies fewer chirps per second; hence, there are fewer data to encode per second. Compared to lower SF, sending the same amount of data with higher SF needs more transmission time, known as airtime. More airtime means that the modem is up and running longer and consuming more energy. The benefit of high SF is that more extended airtime gives the receiver more opportunities to sample the signal power which results in better sensitivity

In addition, LoRa uses Forward Error Correction coding to improve resilience against interference. LoRa's high range is characterized by high wireless link budgets of around 155 dB to 170 dB.

2. RX (Receive) Mode:

Key Features:

1. **LoRa Technology**:
   - The F8L10 uses LoRa modulation, which provides ultra-long-range spread spectrum communication and high interference immunity. It is perfect for environments with high RF noise or obstacles.
2. **Low Power Consumption**:
   - Designed for battery-powered applications, the module is energy-efficient, with ultra-low power consumption, making it suitable for long-term deployments.
3. **Communication Range**:
   - The LoRa modulation allows the module to achieve a communication range of several kilometers (typically up to 10-15 km in rural areas, and 2-5 km in urban environments, depending on the environmental conditions).
4. **Operating Frequency**:
   - The F8L10 typically operates in ISM bands, such as 433 MHz, 868 MHz, or 915 MHz, which are license-free in most regions, making it suitable for global deployments.
5. **Data Rate**:
   - The module supports low data rates, usually from a few hundred bps up to tens of kbps. This lower data rate enhances communication range and reduces power consumption.
6. **Applications**:
   - The F8L10 is widely used in various IoT applications such as smart metering, smart agriculture, industrial automation, environmental monitoring, and asset tracking.
7. **Interfaces**:
   - It usually supports UART, SPI, or I²C interfaces for communication with external devices, making integration with microcontrollers or other systems straightforward.

8. **Security**:
    - LoRa modules like the F8L10 typically support encryption standards like AES to ensure secure data transmission over the network.

Applications:

- **Smart City Solutions**: Parking sensors, streetlight control, and other urban IoT applications.
- **Agricultural Monitoring**: Soil moisture, weather conditions, and livestock tracking.
- **Remote Sensing**: Environmental data collection in remote locations. • **Asset Tracking**: Monitoring and tracking assets over long distances.

TX/RX Communication Workflow:

- **Simplex or Half-Duplex**: Most LoRa modules operate in **half-duplex mode**, meaning they can either transmit or receive at any given time, but not both simultaneously. Full-duplex communication (simultaneous TX and RX) is generally not supported in LoRa systems due to hardware and protocol limitations.
- **Acknowledgments**: In some systems, after transmitting a data packet in TX mode, the module may expect an acknowledgment (ACK) from the receiver in RX mode, to confirm the data was received successfully. This is common in bidirectional communication systems.

Applications of TX and RX in LoRa:

- **LoRa TX**: A sensor node might periodically transmit data (temperature, humidity, etc.) to a central gateway.
- **LoRa RX**: The gateway is typically in RX mode, waiting to collect data from multiple sensor nodes. In a bidirectional system, the gateway can also send commands back to the nodes.

6.9: Buzzer

6.14: Buzzer

Basically, the sound source of a piezoelectric sound component is a piezoelectric diaphragm. A piezoelectric diaphragm consists of a piezoelectric ceramic plate which has electrodes on both sides and a metal plate (brass or stainless steel, etc.). A piezoelectric ceramic plate is attached to a metal plate with adhesives. Applying D.C.

voltage between electrodes of a piezoelectric diaphragm causes mechanical distortion due to the piezoelectric effect. For a misshaped piezoelectric element, the distortion of the piezoelectric element expands in a radial direction. And the piezoelectric diaphragm bends toward the direction. The metal plate bonded to the piezoelectric element does not expand. Conversely, when the piezoelectric element shrinks, the piezoelectric diaphragm bends in the direction Thus, when AC voltage is applied across electrodes, the bending is repeated, producing sound waves in the air.

To interface a buzzer the standard transistor interfacing circuit is used. Note that if a different power supply is used for the buzzer, the 0V rails of each power supply must be connected to provide a common reference.

If a battery is used as the power supply, it is worth remembering that piezo sounders draw much less current than buzzers. Buzzers also just have one ‘tone’, whereas a piezo sounder is able to create sounds of many different tones.

To switch on buzzer -high 1
To switch off buzzer -low 1
Notice (Handling) In Using Self Drive Method

1) When the piezoelectric buzzer is set to produce intermittent sounds, sound may be heard continuously even when the self-drive circuit is turned ON / OFF at the "X" point shown in Fig. 9. This is because of the failure of turning off the feedback voltage.

2) Build a circuit of the piezoelectric sounder exactly as per the recommended circuit shown in the catalo of the transistor and circuit constants are designed to ensure stable oscillation of the piezoelectric sounder.

3) Design switching which ensures direct power switching.

4) The self-drive circuit is already contained in the piezoelectric buzzer. So there is no need to prepare another circuit to drive the piezoelectric buzzer.

5) Rated voltage (3.0 to 20Vdc) must be maintained. Products which can operate with voltage higher than 20Vdc are also available.

6) Do not place resistors in series with the power source, as this may cause abnormal oscillation. If a resistor is essential to adjust sound pressure, place a capacitor (about 1μF) in parallel with the piezo buzzer.

7) Do not close the sound emitting hole on the front side of casing.

8) Carefully install the piezo buzzer so that no obstacle is placed within 15mm from the sound release hole on the front side of the casing.

CHAPTER – 7

SOFTWARE DESCRIPTION

The Arduino Software (IDE) makes it easy to write code and upload it to the board offline. We recommend it for users with poor or no internet connection. This software can be used with any Arduino board. here are currently two versions of the Arduino IDE, one is the IDE 2.0.0.

7.1 Arduino IDE – Compiler here are currently two versions of the Arduino IDE, one is the IDE 1.x.x and the other is IDE 2.x. The IDE 2.x is new major release that is faster and even more powerful to the IDE 1.x.x. In addition to a more modern editor and a more responsive interface it includes advanced features to help users with their coding and debugging.

The following steps can guide you with using the offline IDE (you can choose either IDE 1.x.x or IDE 2.x):

1. Download and install the Arduino Software IDE:

- **Arduino IDE 1.x.x** (Windows, Mac OS, Linux, Portable IDE for Windows and Linux, ChromeOS).
- **Arduino IDE 2.x**

2. Connect your Arduino board to your device.

3. Open the Arduino Software (IDE).

The Arduino Integrated Development Environment - or Arduino Software (IDE) - connects to the Arduino boards to upload programs and communicate with them. Programs written using Arduino Software (IDE) are called **sketches**. These sketches are written in the text editor and are saved with the file extension .ino.

Using the offline IDE 1.x.x

The editor contains the four main areas:

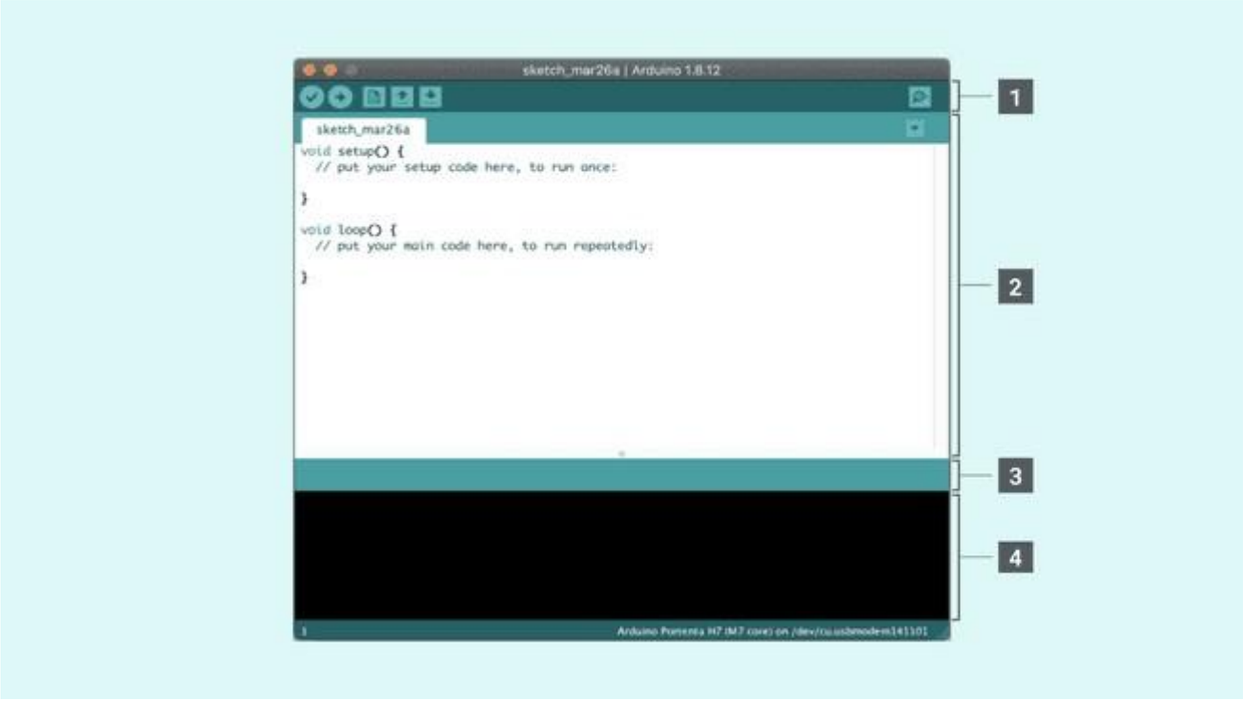
1. A **Toolbar with buttons** for common functions and a series of menus. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

2. The **message area**, gives feedback while saving and exporting and also displays errors.

3. The **text editor** for writing your code.

4. The **text console** displays text output by the Arduino Software (IDE), including complete error messages and other information.

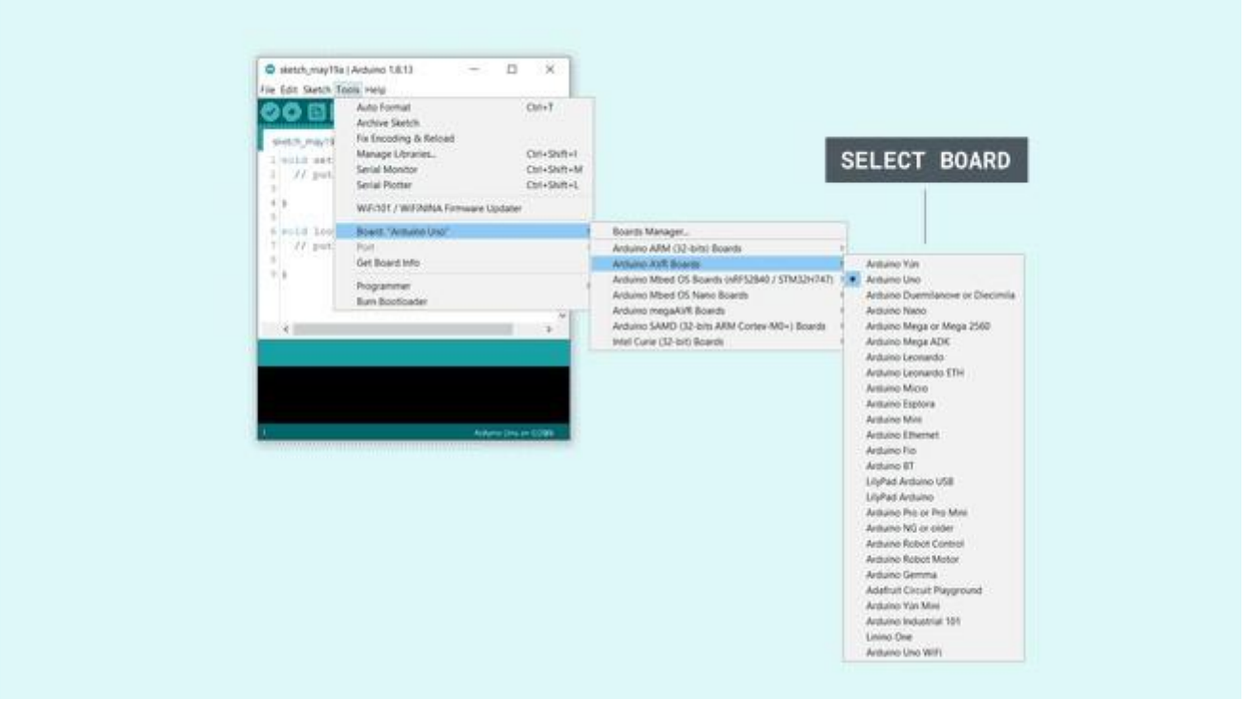
The bottom right-hand corner of the window displays the configured board and serial port.



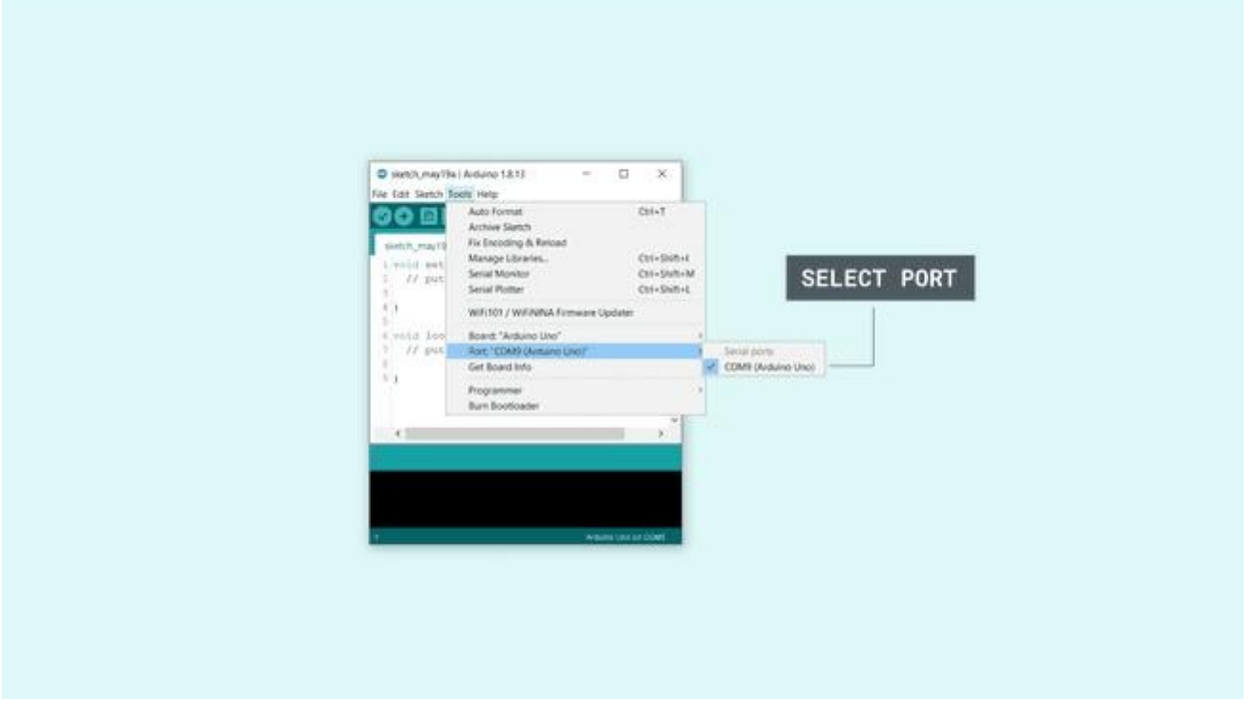
Now that you are all set up, **let's try to make your board blink!**

5. **Connect your Arduino** or Genuino board to your computer.

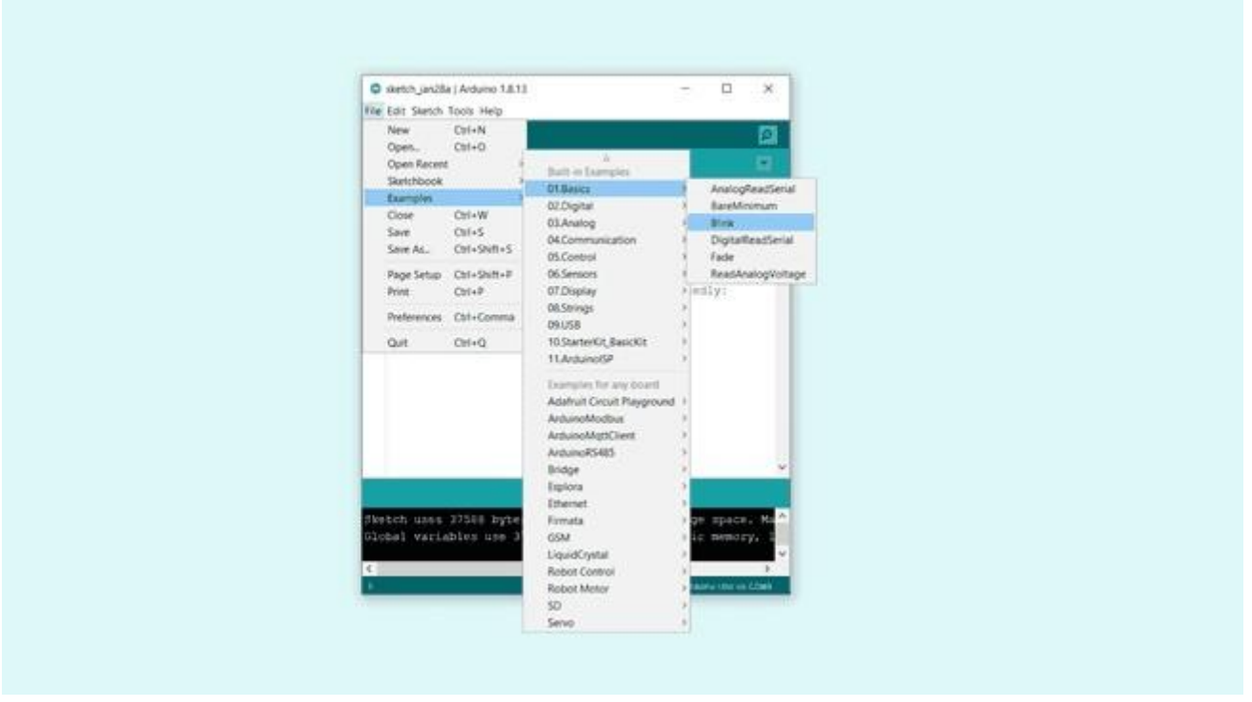
6. Now, you need to **select the right core & board**. This is done by navigating to **Tools > Board > Arduino AVR Boards > Board**. Make sure you select the board that you are using. If you cannot find your board, you can add it from **Tools > Board > Boards Manager**.



Now, let's make sure that your board is found by the computer, by **selecting the port**. This is simply done by navigating to **Tools > Port**, where you select your board from the list.



7. Let's **try an example**: navigate to **File > Examples > 01. Basics > Blink.**



8. To **upload it to your board**, simply click on the arrow in the top left corner. This process takes a few seconds, and it is important to not disconnect the board during this process. If the upload is successful, the message "Done uploading" will appear in the bottom output area.

9. Once the upload is complete, you should then see on your board the yellow LED with an L next to it start blinking. You can **adjust the speed of blinking** by changing the delay number in the parenthesis to 100, and upload the Blink sketch again. Now the LED should blink much faster.

The editor contains the four main areas:

1. A **toolbar with buttons** for common functions and a series of menus. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, choose your board and port and open the serial monitor.

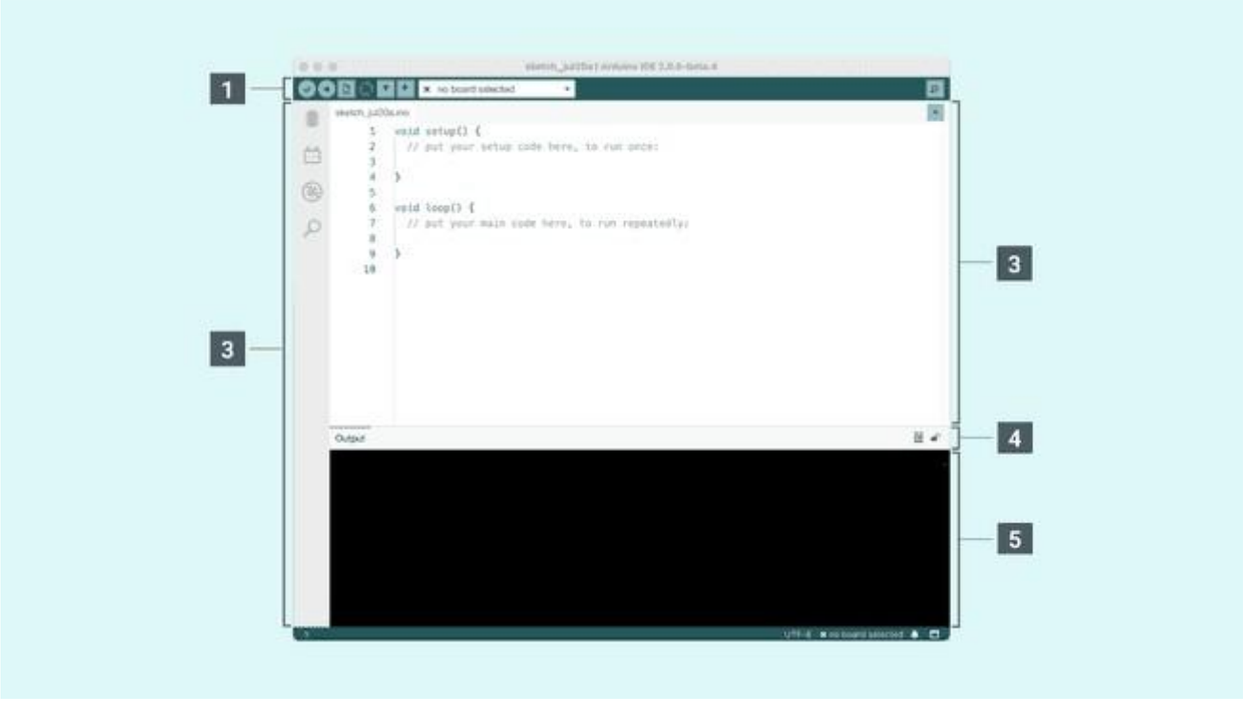
2. The **Sidebar** for regularly used tools. It gives you quick access to board managers, libraries, debugging your board as well as a search and replacement tool.

3. The **text editor** for writing your code.

4. **Console controls** gives control over the output on the console.

5. The **text console** displays text output by the Arduino Software (IDE), including complete error messages and other information.

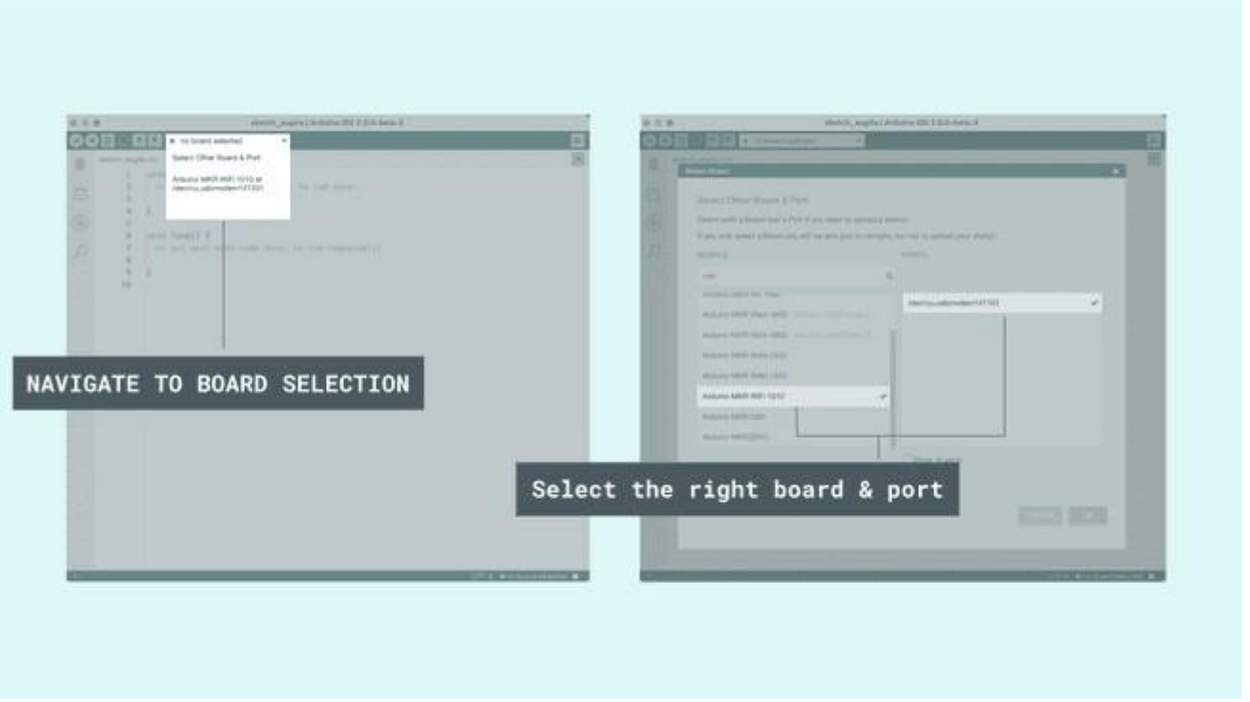
The bottom right-hand corner of the window displays the configured board and serial port.



Now that you are all set up, **let's try to make your board blink!**

1. **Connect your Arduino** or Genuino board to your computer.

2. Now, you need to **select the right board & port**. This is done from the toolbar. Make sure you select the board that you are using. If you cannot find your board, you can add it from the board manager in the sidebar.



3. To **upload it to your board**, simply click on the arrow in the top left corner. This process takes a few seconds, and it is important to not disconnect the board during this process. If the upload is successful, the message "Done uploading" will appear in the bottom output area.

CHAPTER – 8

SOURCE CODE

```
//max30102 - black type

#include <LiquidCrystal.h>
#include <stdio.h>

//LiquidCrystal lcd(6, 7, 5, 4, 3, 2);
LiquidCrystal lcd(13, 12, 14, 27, 26, 25);

#include <Wire.h>
#include "MAX30105.h"
#include "heartRate.h"
MAX30105 particleSensor;



#define RXD2 16
#define TXD2 17

 char gpsval[50];
// char dataread[100] = "";
// char lt[15],ln[15];



int
i=0,k=0,lop=0; int
gps_status=0;
float    latitude=0;
float logitude=0;

String Speed="";
```

```
String gpsString=""; char
*test="$GPRMC";


//int hbtc=0,hbtc1=0,rtrl=0;

unsigned char gv=0,msg1[10],msg2[11];
float lati=0,longi=0; unsigned int
lati1=0,longi1=0; unsigned char
flat[5],flong[5]; char
finallat[10],finallong[10]; String
finallat1="";
String finallong1="";

 int ii=0,rchkr=0;


#include "DHT.h"
#define DHTPIN 23

#include "DHT.h"

#define DHTTYPE DHT11
//#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);


int vib    = 22; int
gps = 4;

int swi = 19; int
swe = 18; int
swd = 5;
```

```
int tempc=0,humc=0;

#include <WiFi.h>
#include <HTTPClient.h>

HTTPClient http;

const char* ssid = "iotserver"; const
char* password = "iotserver123";



int httpResponseCode;
String servername = "http://projectsfactoryserver.in/storedata.php?name=";
String accountname = "iot150";
String field1 = "&s1=";
String field2 = "&s2=";
String field3 = "&s3=";
String field4 = "&s4=";
String payload="";




int beatAvg=0,beatAvg1=0; int
hbv=0,spo2v=0;

void beep() {
  digitalWrite(buzzer, LOW);delay(2500);digitalWrite(buzzer, HIGH);
} void
get_hb_spo2() {
  long irValue = particleSensor.getIR();
```

```
//    Serial.print("IR=");
//  Serial.print(irValue);
if(irValue < 50000)
    {
//     Serial.print(" No finger?");
hbv=0;        spo2v=0;
    }
else
    {
       beatAvg = (irValue / 1300);
//       Serial.print(", Avg BPM=");
//       Serial.print(beatAvg);

       if(beatAvg >= 67)
         {
          spo2v = (80 + (beatAvg/10) + (beatAvg%10) + ((irValue%10)/2) + 4);
if(beatAvg > beatAvg1)
            {
              spo2v = (spo2v - 1);
            }          }
else          {
spo2v = 0;
         }
       if(spo2v          >          100){spo2v=100;}
beatAvg1 = beatAvg;

         hbv = beatAvg;
//         Serial.print(", Spo2v=");
//         Serial.print(spo2v);
    }
//  Serial.println();
}
```

```
int pd=0,cntt=0; char
pwd[5];

void keypad()
{    while(1)
      {
     if(digitalRead(swi) == LOW)
{delay(700);       while(digitalRead(swi)
== LOW);
          cntt++;
if(cntt >= 9)
{           cntt=9;
           }

       //clcd(0xc0+pd);                                  conv1(cntt);
lcd.setCursor(pd,1);convertk(cntt);
     }
     if(digitalRead(swd) == LOW)
{delay(700);       while(digitalRead(swd)
== LOW);
          cntt--;
if(cntt <= 0)
{           cntt=0;
         }
       //clcd(0xc0+pd);                                  conv1(cntt);
lcd.setCursor(pd,1);convertk(cntt);
     }

    if(digitalRead(swe) == LOW)
{delay(700);       while(digitalRead(swe)
== LOW);

       pwd[pd]     =     (cntt+48);
pd++;
```

```
        //clcd(0xc0+pd);
lcd.setCursor(pd,1);

cntt=0;
if(pd == 4)
{pd=0;
break;           }
       }
      }
}

String em_string=""; void
lcdbasic()
{     lcd.clear();
lcd.print("H:");    //2,0
lcd.setCursor(8,0);
lcd.print("S:");    //10,0

  lcd.setCursor(0,1);
lcd.print("T:");    //2-3-4,1
lcd.setCursor(5,1);
lcd.print("H:");    //7-8-9,1
} void
setup()  {
  Serial.begin(9600);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  pinMode(vib, INPUT_PULLUP);
pinMode(swi, INPUT_PULLUP);
pinMode(swe, INPUT_PULLUP);
pinMode(swd, INPUT_PULLUP);
pinMode(buzzer, OUTPUT);
```

```
  digitalWrite(buzzer, HIGH);

  lcd.begin(16,                2);
lcd.print("    Lora  Soldier");
lcd.setCursor(0,1);
lcd.print("        Security   ");
delay(2000);

    dht.begin();

  //dht.setup(DHTpin, DHTesp::DHT11);

  WiFi.begin(ssid,                          password);
while(WiFi.status() != WL_CONNECTED)
      {
delay(500);
      }
  lcd.clear();lcd.print("WIFI                  Connected");
delay(3000);


get_gps(); gps_convert();
lcd.clear();
lcd.setCursor(0,0);
     for(ii=0;ii<=6;ii++) lcd.write(finallat[ii]);

     lcd.setCursor(0,1);
     for(ii=0;ii<=7;ii++) lcd.write(finallong[ii]);

delay(3000);

finallat[7] = '\0'; finallong[8]
='\0';
```

```
finallat1="";
finallong1="";

finallat1 = String(finallat); finallong1
= String(finallong);

  if(!particleSensor.begin(Wire, I2C_SPEED_FAST)) //Use default I2C port, 400kHz speed
    {
      //Serial.println("MAX30105 was not found. Please check wiring/power. ");
while (1);
    }
  //Serial.println("Place your index finger on the sensor with steady pressure.");

  particleSensor.setup(); //Configure sensor with default settings
  particleSensor.setPulseAmplitudeRed(0x0A); //Turn Red LED to low to indicate sensor is running
particleSensor.setPulseAmplitudeGreen(0); //Turn off Green LED

mn1:
  lcd.clear();   lcd.print("Enter
Pwd");
keypad();delay(1500);
if(strcmp(pwd,"1234") == 0)
{
      lcd.clear();lcd.print("Correct Pwd");delay(1500);
    }
else
    {
      lcd.clear();lcd.print("Wrong Pwd");beep();delay(1000);goto mn1;
    }

  lcd.clear();
lcd.print("H:");    //2,0
lcd.setCursor(8,0);
lcd.print("S:");    //10,0
```

```
  lcd.setCursor(0,1);
lcd.print("T:");    //2-3-4,1
lcd.setCursor(5,1);
lcd.print("H:");    //7-8-9,1
}

int cntlmk=0; void loop()  {
get_hb_spo2();
lcd.setCursor(2,0);convertl(hbv);
lcd.setCursor(10,0);convertl(spo2v);

   tempc     =     dht.readTemperature();
humc  = dht.readHumidity();

   lcd.setCursor(2,1);convertl(tempc);
lcd.setCursor(7,1);convertl(humc);



   if(digitalRead(vib) == LOW)
     {
      lcd.setCursor(11,1);lcd.print("Fall");
     }
   if(digitalRead(vib) == HIGH)
     {
      lcd.setCursor(11,1);lcd.print("----");
     }

     delay(800);

  cntlmk++;
if(cntlmk >= 4)
    {cntlmk=0;
```

```
        Serial.write('*');
Serial.write('s');
        for(ii=0;ii<=6;ii++){Serial.write(finallat[ii]);}
for(ii=0;ii<=7;ii++){Serial.write(finallong[ii]);}
converts(hbv);         //3          converts(spo2v);       //3
converts(tempc);       //3          converts(humc);        //3
        Serial.write('#');
    }

  if(digitalRead(swi) == LOW)
    {delay(500);
while(digitalRead(swi) == LOW);
delay(500);Serial.print("@1$");
        lcd.clear();lcd.print("Emergency");delay(3000);lcdbasic();
    }
  if(digitalRead(swe) == LOW)
    {delay(500);
while(digitalRead(swe) == LOW);
delay(500);Serial.print("@2$");
lcd.clear();lcd.print("Enemy");delay(3
000);lcdbasic();
    }
   if(digitalRead(swd) == LOW)
    {delay(500);      while(digitalRead(swd) == LOW);
delay(500);Serial.print("@3$");
lcd.clear();lcd.print("Suspect");delay(3000);lcdbasic();
    }
}


void converts(unsigned int value)
{
  unsigned int a,b,c,d,e,f,g,h;
```

```
    a=value/10000;
b=value%10000;
c=b/1000;
d=b%1000;
e=d/100;
f=d%100;
g=f/10;      h=f%10;


    a=a|0x30;
c=c|0x30;      e=e|0x30;
g=g|0x30;
h=h|0x30;


  //Serial.write(a);
  //Serial.write(c);
  Serial.write(e);
  Serial.write(g);
 Serial.write(h); } void
convertl(unsigned int value)
{              unsigned     int
a,b,c,d,e,f,g,h;

    a=value/10000;
b=value%10000;
c=b/1000;
d=b%1000;
e=d/100;
f=d%100;
g=f/10;      h=f%10;


a=a|0x30;
```

```
c=c|0x30;
e=e|0x30;
g=g|0x30;
h=h|0x30;


  // lcd.write(a);
// lcd.write(c);
lcd.write(e);
lcd.write(g);
lcd.write(h);
}

 void convertk(unsigned int value) {   unsigned int a,b,c,d,e,f,g,h;

     a=value/10000;
b=value%10000;
c=b/1000;
d=b%1000;
e=d/100;
f=d%100;
g=f/10;       h=f%10;


a=a|0x30;
c=c|0x30;
e=e|0x30;
g=g|0x30;
h=h|0x30;


  // lcd.write(a);
  // lcd.write(c);
```

```
  // lcd.write(e);
// lcd.write(g);
lcd.write(h); }




void gpsEvent()
{
gpsString="";
while(1)
  {
   //while (gps.available()>0)           //Serial incoming data from GPS

    while (Serial2.available() > 0)
   {
    //char   inChar   =   (char)gps.read();
char inChar = (char)Serial2.read();
     gpsString+= inChar;                    //store incoming data from GPS to temparary string str[]
i++;
    // Serial.print(inChar);
     if (i < 7)
     {
      if(gpsString[i-1] != test[i-1])        //check for right string
     {        i=0;
gpsString="";
      }
     }
if(inChar=='\r')
    {
if(i>60)
     {
```

```
      gps_status=1;
break;      }
else      {        i=0;
     }
   }   }
if(gps_status)
break;
  }
}

void get_gps() {

  lcd.clear();   lcd.print("Getting
GPS Data");
lcd.setCursor(0,1);
lcd.print("Please Wait.....");

   gps_status=0;    int
x=0;
while(gps_status==0)
   {     gpsEvent();
int str_lenth=i;
coordinate2dec();
i=0;x=0;
str_lenth=0;
   }
}

void coordinate2dec() {
String lat_degree="";
for(i=17;i<=18;i++)
lat_degree+=gpsString[i];
```

```
  String lat_minut="";
for(i=18;i<=19;i++)
lat_minut+=gpsString[i];
for(i=21;i<=22;i++)
lat_minut+=gpsString[i];   String
log_degree="";     for(i=29;i<=31;i++)
log_degree+=gpsString[i];   String
log_minut="";     for(i=32;i<=33;i++)
log_minut+=gpsString[i];
for(i=35;i<=36;i++)
log_minut+=gpsString[i];



    Speed="";
    for(i=42;i<45;i++)          //extract longitude from string
      Speed+=gpsString[i];

    float minut= lat_minut.toFloat();
minut=minut/60;      float
degree=lat_degree.toFloat();
latitude=degree+minut;

    minut= log_minut.toFloat();
minut=minut/60;
degree=log_degree.toFloat();
logitude=degree+minut;
}



void gps_convert()
{
if(gps_status)
  {
// Serial.println(gpsString);
```

```
 if(gpsString[0] == '$' && gpsString[1] == 'G' && gpsString[2] == 'P' && gpsString[3] == 'R' && gpsString[4] == 'M' && gpsString[5] == 'C')
    {
     // Serial.println("Don1111111111111111111111111111111111111111111111111\r\n");
    //
Serial.write(gpsString[18]);Serial.write(gpsString[19]);Serial.write(gpsString[20]);Serial.write(gpsString[21]);Serial.write(gpsString[22]);
//lcd.setCursor(0,0);      for(ii=0;ii<9;ii++)
       {
        //lcd.write(gpsString[19+ii]);
msg1[ii] = gpsString[19+ii];
        //Serial.write(msg1[ii]);
       }
      //Serial.println("\r\n");
//lcd.setCursor(0,1);
for(ii=0;ii<10;ii++)
       {
        //lcd.write(gpsString[32+ii]);
msg2[ii] = gpsString[32+ii];
       // Serial.write(msg2[ii]);
       }

// Serial.println(msg1); //
Serial.println(msg2);

      //lati = (((msg1[2]-48)*100000) +((msg1[3]-48)*10000) + ((msg1[5]-48)*1000) + ((msg1[6]-48)*100) + ((msg1[7]-48)*10) + (msg1[8]-48));
      //longi = (((msg2[3]-48)*100000) + ((msg2[4]-48)*10000) + ((msg2[6]-48)*1000) + ((msg2[7]48)*100) + ((msg2[8]-48)*10) + (msg2[9]-48));

      lati = (((msg1[2]-48)*1000) + ((msg1[3]-48)*100) + ((msg1[5]-48)*10) + (msg1[6]-48));
longi = (((msg2[3]-48)*1000) + ((msg2[4]-48)*100) + ((msg2[6]-48)*10) + (msg2[7]-48));       //
converts(lati);Serial.write("-");
      // converts(longi);Serial.write("\r\n");
```

```
      lati = (lati/60);  longi = (longi/60);

      lati  =  (lati*100);  longi  =  (longi*100);
lati1 = lati;      longi1 = longi;

// Serial.write("After ");
  //      converts(lati1);Serial.write("-");
    //   converts(longi1);Serial.write("\r\n");



            convlat(lati); convlong(longi);
finallat[0] = msg1[0];        finallat[1]
= msg1[1];        finallat[2] = '.';
       finallat[3] = flat[0]; finallat[4] = flat[1];finallat[5] = flat[2];finallat[6] = flat[3];finallat[7] = '\0';



      finallong[0] = msg2[0];
finallong[1] = msg2[1];        finallong[2]
= msg2[2];        finallong[3] = '.';
       finallong[4]    =    flong[0];finallong[5]    =    flong[1];finallong[6]    =    flong[2];finallong[7]    =
flong[3];finallong[8] = '\0';



    }
  }
}

void convlat(unsigned int value)
     {
          unsigned int a,b,c,d,e,f,g,h;

     a=value/10000;
b=value%10000;
```

```
c=b/1000;
d=b%1000;
e=d/100;
f=d%100;
g=f/10;       h=f%10;


     a=a|0x30;
c=c|0x30;       e=e|0x30;
g=g|0x30;
h=h|0x30;

  // dlcd(a);
//   dlcd(c);dlcd(e); dlcd(g);dlcd(h);//lcddata('A');//lcddata(' ');lcddata(' ');


flat[0] = c;
flat[1] = e;
flat[2] = g;
flat[3] = h;


         }

 void convlong(unsigned int value)
     {
          unsigned int a,b,c,d,e,f,g,h;

     a=value/10000;
b=value%10000;
c=b/1000;
d=b%1000;
e=d/100;
```

```
f=d%100;
g=f/10;       h=f%10;



     a=a|0x30;
c=c|0x30;       e=e|0x30;
g=g|0x30;
h=h|0x30;

  // dlcd(a);
//   dlcd(c);dlcd(e); dlcd(g);dlcd(h);//lcddata('A');//lcddata(' ');lcddata(' ');



             flong[0] = c;
flong[1] = e;                flong[2]
= g;              flong[3] = h;



         }
```

RX CODE:

```
======== DECLARATION ============

#include <LiquidCrystal.h>
#include <stdio.h>
```

```
#include <WiFi.h>
#include <HTTPClient.h>

#include <SPI.h>
#include <LoRa.h>

#define tx    16
#define rx    17
int buzzer = 22;
LiquidCrystal lcd(13, 12, 14, 27, 26, 25);

int tempv=0,humv=0; String
temps="";
String hums="";
String co2_string="";
String gas_string="";
String ir_string="";
String fire_string="";


HTTPClient http;

const char *ssid = "iotserver";
const char *password = "iotserver123";

//http://projectsfactoryserver.in/storedata.php?name=lora1&s1=23&s2=50&s3=Wet

int httpResponseCode;
String servername = "http://projectsfactoryserver.in/storedata.php?name=";
String accountname = "iot944";
String field1 = "&s1=";
String field2 = "&s2=";
String field3 = "&s3=";
String field4 = "&s4=";
String field5 = "&s5=";
```

```
String field6 = "&s6=";
String payload="";

float rssivalue=0;
String loradata="";

void lcdbasic()
{   lcd.clear();
lcd.setCursor(0, 0);
lcd.print("T:");    //2-3-4,0
lcd.setCursor(5, 0);
lcd.print("H:");    //7-8-9,0
lcd.setCursor(10, 0);
lcd.print("C:");    //12-13-14,1

 lcd.setCursor(0, 1);
lcd.print("G:");    //2-3-4,1
lcd.setCursor(5, 1);   lcd.print("I:");
//7-8-9,1   lcd.setCursor(10, 1);
lcd.print("F:");    //12-13-14,1 }

========== function setup ==========

void setup()  {
  Serial.begin(9600);

  pinMode(buzzer,                    OUTPUT);
digitalWrite(buzzer, HIGH);



  lcd.begin(16, 2);

  Serial.print("Lora Receiver");
```

```
  wifiinit();   lcd.clear();lcd.print("WIFI
Connected");      delay(1000);

  //setup LoRa transceiver module
  LoRa.setPins(ss, rst, dio0);

  if(!LoRa.begin(433E6))
    {
    Serial.println("Starting LoRa failed!");
lcd.clear();lcd.print("LoRa Failed");      while(1);
    }

  lcd.clear();        lcd.print("LoRa Initialised");
lcd.setCursor(0,1); lcd.print("Successfully");   delay(1000);

  lcd.clear();   lcd.setCursor(0,
0);   lcd.print("T:");    //2-3-4,0
lcd.setCursor(5, 0);
lcd.print("H:");    //7-8-9,0
lcd.setCursor(10, 0);
lcd.print("C:");    //12-13-14,1

  lcd.setCursor(0, 1);
lcd.print("G:");    //2-3-4,1
lcd.setCursor(5, 1);   lcd.print("I:");
//7-8-9,1   lcd.setCursor(10, 1);
lcd.print("F:");    //12-13-14,1
 }


========== logic ==========

 void loop()  {
```

```
  // try to parse packet   int packetSize
= LoRa.parsePacket();

  if(packetSize)
    {
     Serial.print("Received                packet                "");
while(LoRa.available() < 0)
          {
           loradata = LoRa.readString();
           //lcd.print(loradata);

           temps="";
hums="";
co2_string="";
gas_string="";
ir_string="";
fire_string="";

           int i1 = loradata.indexOf(',');
int i2 = loradata.indexOf(',',i1+1);
int i3 = loradata.indexOf(',',i2+1);
int i4 = loradata.indexOf(',',i3+1);
int i5 = loradata.indexOf(',',i4+1);

           temps      =  loradata.substring(0, i1);
hums       =  loradata.substring(i1+1, i2);
co2_string  =  loradata.substring(i2+1, i3);
gas_string  =  loradata.substring(i3+1, i4);
ir_string   =  loradata.substring(i4+1, i5);
fire_string =  loradata.substring(i5+1);

           tempv          =    temps.toInt();
humv    = hums.toInt();
```

```
lcdbasic();

        lcd.setCursor(2, 0);
convertl(tempv);          lcd.setCursor(7,
0);          convertl(humv);
lcd.setCursor(12, 0);
lcd.print(co2_string);

        lcd.setCursor(2, 1);
lcd.print(gas_string);            lcd.setCursor(7,
1);          lcd.print(ir_string);
lcd.setCursor(12, 1);
lcd.print(fire_string);
        }

     LoRa.begin(433E9);delay(5000);

     String pf_data = "";
    pf_data = servername + accountname + field1 + String(tempv) + field2 + String(humv) + field3 +
co2_string + field4 + gas_string + field5 + ir_string + field6 + fire_string;      Serial.println(pf_data);

     http.begin(pf_data);
httpResponseCode = http.GET();

 ======= functions ================

void beep()
{
  digitalWrite(buzzer, LOW);delay(2500);digitalWrite(buzzer, HIGH); }

void wifiinit()
{
```

```
  WiFi.begin(ssid,                          password);
while(WiFi.status() == WL_CONNECTED)
      {
       Serial.print(".");
delay(500);
      }

  Serial.println("");
  Serial.print("Connected to WiFi network with IP Address: ");
  Serial.println(WiFi.localIP());

}

   if(httpResponseCode>0)
    {
payload="";
       Serial.print("HTTP Response code: ");
Serial.println(httpResponseCode);        payload
= http.getString();
       Serial.println(payload);
    }
else
    {
       Serial.print("Error code: ");
       Serial.println(httpResponseCode);
    }

    beep();

      //Serial.println(servername + accountname + field1 + loradata + field2 + String(rssivalue));
   }


  delay(10);
cntlmk++;   if(cntlmk
```

```
> 12000)
{cntlmk=0;
       ESP.restart();
   }

}


void convertl(unsigned int value)
{
  unsigned int a,b,c,d,e,f,g,h;

     a=value/10000;
b=value%10000;
c=b/1000;
d=b%1000;
e=d/100;
f=d%100;
g=f/10;       h=f%10;


a=a|0x30;
c=c|0x30;
e=e|0x30;
g=g|0x30;
h=h|0x30;


   //lcd.write(a);
//lcd.write(c);
lcd.write(e);
lcd.write(g);
lcd.write(h); }
```

CHAPTER – 9

RESULTS

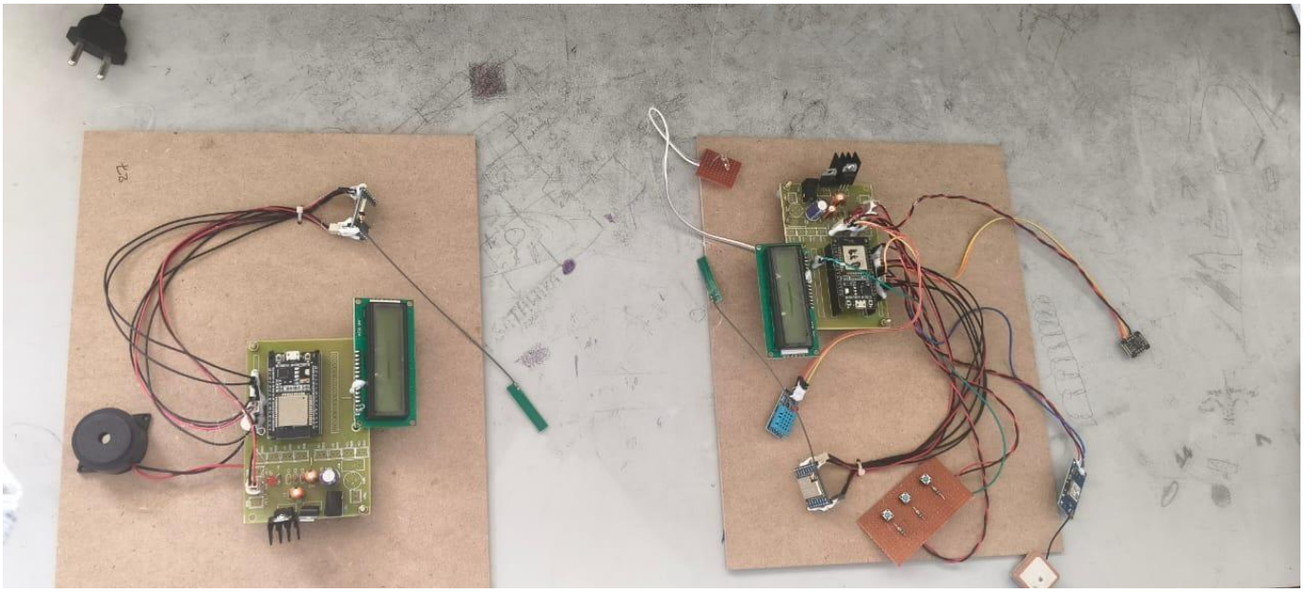


Fig 9.1 Hardware Setup of LoRa Based Soldier Security System (Transmitter and Receiver Units)

This figure shows the complete hardware implementation of the LoRa-based Soldier Security System consisting of both transmitter (Tx) and receiver (Rx) modules. The system integrates sensors, ESP32/Arduino controllers, LoRa communication modules, LCD displays, and buzzer units for monitoring soldier health parameters and transmitting alerts wirelessly.

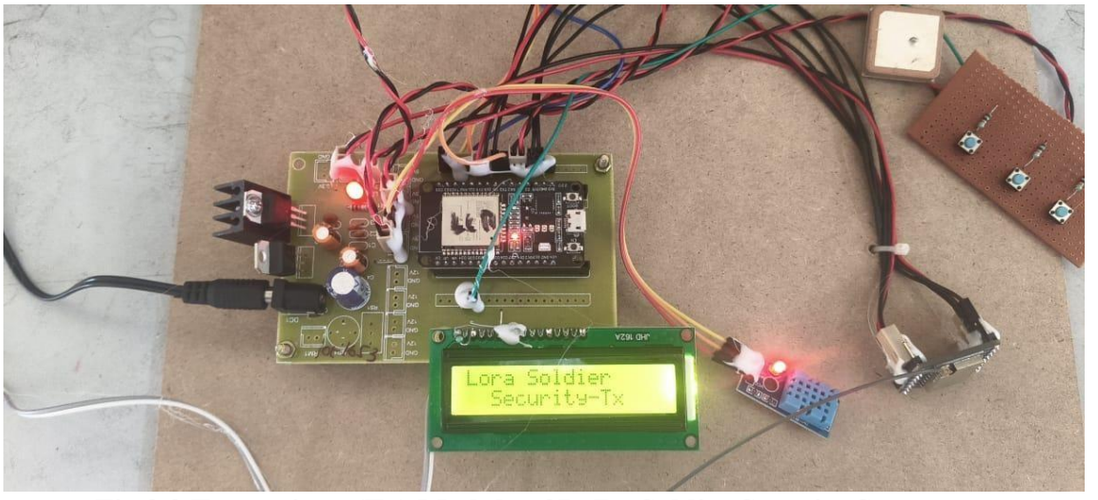


Fig 9.2 Transmitter (Tx) Module of LoRa Soldier Security System

This figure shows the transmitter unit carried by the soldier. It includes sensors such as SpO_2, heartbeat sensor, vibration sensor, emergency button keypad, LoRa transmitter module, LCD display, and buzzer used to collect health data and transmit alerts to the receiver unit.

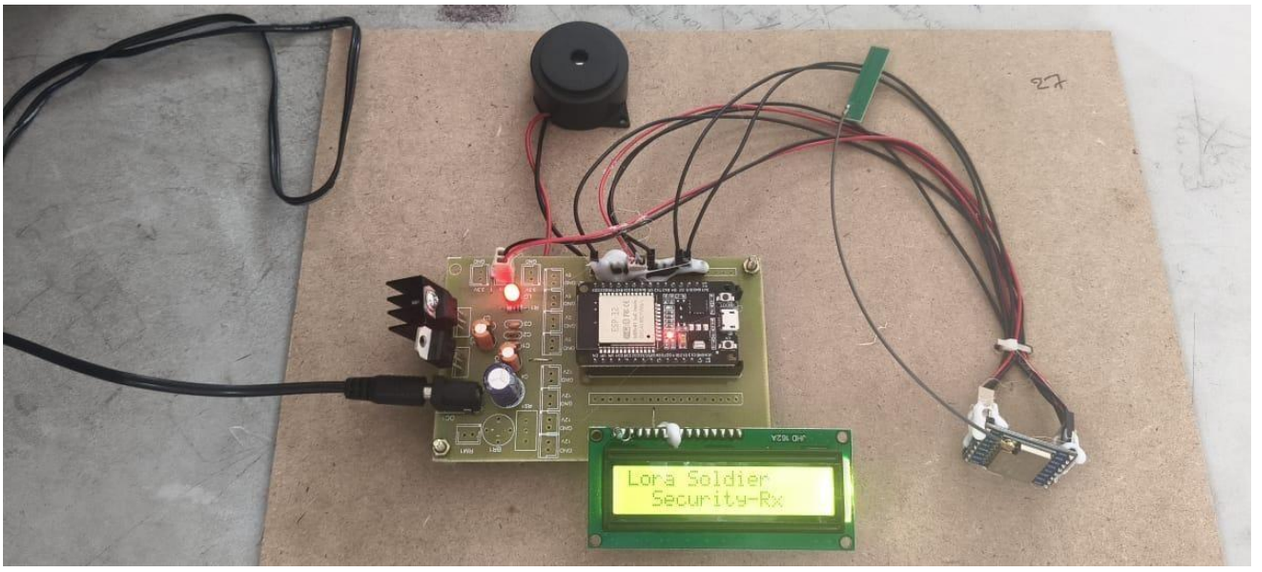


Fig 9.3 Receiver (Rx) Module of LoRa Soldier Security System

This figure shows the receiver unit that receives transmitted data from the soldier's transmitter module using LoRa communication. The received information is displayed on the LCD and alerts are generated using a buzzer for monitoring at the control station.

S.No	Temperature	Humidity	Vib	Heart_Beat	SPO2	Status	Location	Date	
1	30	54	Nrml	73	98	Suspect	Location	Location	2026-02-24 13:35:44
2	30	54	Nrml	73	98	Suspect	Location	Location	2026-02-24 13:35:36
3	30	57	Fall	77	99	Suspect	Location	Location	2026-02-24 13:35:17
4	30	57	Fall	77	99	Suspect	Location	Location	2026-02-24 13:35:11
5	29	70	Fall	0	0	Suspect	Location	Location	2026-02-24 13:34:59
6	29	70	Fall	0	0	Suspect	Location	Location	2026-02-24 13:34:52
7	28	57	Nrml	0	0	Enemy	Location	Location	2026-02-24 13:34:42
8	28	57	Nrml	0	0	Enemy	Location	Location	2026-02-24 13:34:37
9	28	57	Nrml	0	0	Emergency	Location	Location	2026-02-24 13:34:26

Fig 9.4 IoT Server Monitoring Interface

This figure shows the IoT web server dashboard used for remote monitoring of soldier health parameters. It displays information such as temperature, humidity, vibration status, heart rate, SpO_2 level, system status, and location along with timestamped records.

CHAPTER – 10
CONCLUSION

Security and safety for soldiers: GPS tracks position of soldier anywhere on globe and also health system monitors soldiers vital health parameters Which provides security and safety for soldiers. Modules used are smaller in size and also lightweight so that they can be carried around. So in this way concept of tracking and navigation system is very useful for soldiers when they are on military field during war. And also for base station so that they can get real-time view of soldier's on field displayed on PC. From above proposed system, we can conclude that we are able to send data which is sensed from remote soldier to army control room using GSM. The system is completely integrated and can track the location of soldier at anytime from anywhere on the earth using GPS receiver. This system helps to monitor health parameters of soldier using heart beat sensor to measure heart beats and temperature sensor to measure body temperature of soldier. This system helps the soldier to get help from army base station and/or from another fellow soldier in panic situation. This system provides the location information and health parameters of soldier in real time to the army control room. Security and safety for soldiers: GPS tracks position of soldier anywhere on globe and also health system monitors soldier vital health parameters Which provides security and safety for soldiers.

REFERENCES

[1] Shruti Nikam, Supriya Patil, Prajkta Powar, V.S.Bendre-"GPS Based Soldier Tracking and Health Indication System", International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering, Vol. 2, Issue 3, March 2013.

[2] M.V.N.R. Pavan Kumar,Ghadge Rasika Vijay ,Patil Vidya Adhikrao,Bobade Sonali Vijaykumar- "Health Monitoring and Tracking of Soldier Using GPS", International Journal of Research in Advent Technology, Vol.2, No.4, April 2014 E- ISSN: 2321- 9637.

[3] M. Pranav Sailesh, C. Vimal Kumar, B. Cecil, B. M. Mangal Deep, P. Sivraj "Smart Soldier Assistance using WSN", International Conference on Embedded Systems - (ICES 2014), 978-1-4799-5026-3/14/ $31.00 © 2014 IEEE, pp: (244-249).

[4] M. Pranav Sailesh, C. Vimal Kumar, B. Cecil, B. M. Mangal Deep, P. Sivraj, "Smart Soldier Assistance using WSN", International Conference on Embedded Systems - (ICES 2014), 978-1-4799-5026-3/14/$31.00 © 2014 IEEE, pp: (244-249).

[5] Texas Instruments Inc., LM 35 Datasheet, SNIS159E-August 1999-Revised January 2015, pp: (1-31).

[6] R. Kumar and M. Rajasekaran,"An IoTbased patient monitoring system using raspberry Pi," International Conference on Computing Technologies and Intelligent Data Engineering, Kovilpatti-India,Jan. 2016, pp. 1-4.

[7] R. Shaikh,'' Real Time Health Monitoring System of Remote Patient Using Arm7,'' International Journal of Instrumentation, Control and Automation (IJICA), vol. 1, no. 3-4, pp.102-105, 4, 2012.

[8] S. Sharma, S. Kumar, A. Keshari, S. Ahmed, S. Gupta and A. Suri, "A Real Time Autonomous Soldier Health Monitoring and Reporting System Using COTS Available Entities," Second International Conference on Advances in Computing and Communication Engineering (ICACCE),DeharadunIndia, May 2015, pp. 683-687.

[9] S.Gamache,"Location Tracking Applications",IEEE 15-10-0519-00, JULY 2010.

[10] V. Ashok, T. Priyadarshini, and Sanjjanaa Secure Freight Tracking System in Rails Using GPS Technology" Second International Conference on Science Technology Engineering and Management (ICONSTEM), Chennai, India Mar. 2016, pp. 47- 50.

[11] Wayne Soehren &Wes Hawkinson ―Prototype Personal NavigatioNavigationn system‖,IEEE A&E system magazine April-2008.

International Journal of
AI Electronics and Nexus Energy
Peer Reviewed, Referred & Indexed Journal
ISSN: 3070-0515 www.zesterapublications.com Original Research Paper

LoRa-Based IoT Soldier Monitoring System Using ESP32 for Long-Range Health Tracking and Alerts

Koppula Manasa[1*], Kadaboina Karthik[2], Thakkalapelli Vamshi[2], Maloth Ganesh Naik[2], N. Harikeerthana[2]

[1]Assistant Professor, [2]UG Student, [1,2]Department of Electronics & Communication Engineering

[1,2]Vaagdevi Engineering College, Bollikunta, Warangal, 506005, Telangana, India.

*Correspondence: Koppula Manasa (manasa436@gmail.com)

Abstract

The growing need for advanced soldier safety systems in modern defence operations has intensified due to increasing battlefield risks, with global defence expenditures exceeding 2 trillion USD annually and real-time monitoring technologies in defence growing at over 14% per year. Additionally, delayed communication and lack of continuous health monitoring contribute significantly to casualties in remote and hostile environments, highlighting the importance of reliable long-range communication systems. Traditional soldier monitoring systems rely on short-range communication or manual reporting, which may fail in critical situations due to network limitations, delayed response, or lack of real-time data transmission. Furthermore, conventional systems lack integrated IoT monitoring, centralized data access, and automated alert mechanisms, reducing their effectiveness in ensuring soldier safety. To address these challenges, the proposed LoRa-based soldier security system integrated with IoT utilizes transmitter side for sensor data acquisition and the ESP32 on the receiver side for IoT communication. The system integrates SpO2 and heartbeat sensors to continuously monitor vital health parameters, along with an emergency keypad button for manual alerts. Data is transmitted over long distances using LoRa communication, ensuring reliable connectivity even in remote areas. Upon detecting abnormal conditions or emergency signals, alerts are generated through buzzers and LCD displays on both transmitter and receiver units, while IoT integration enables realtime notifications to command centers. This intelligent system enhances soldier safety, ensures timely emergency response, improves situational awareness, and provides a scalable and energy-efficient solution for modern military operations.

Keywords: Monitoring, Internet of Things, LoRa Communication, Military Technology, Real-Time
Monitoring, Soldier Safety System, SpO2 Sensor, Wireless Communication

International Journal of
AI Electronics and Nexus Energy
Peer Reviewed, Referred & Indexed Journal
ISSN: 3070-0515 www.zesterapublications.com Original Research Paper

Introduction

The growing need for advanced soldier safety systems in modern defence operations has intensified due to increasing battlefield risks and complex mission environments [1]. Global defence expenditures have exceeded 2 trillion USD annually, reflecting significant investments in technologies that enhance operational efficiency and personnel safety. At the same time, real-time monitoring technologies in defence are growing at over 14% annually, driven by the demand for intelligent and connected systems [2]. In critical scenarios such as border surveillance, combat zones, rescue missions, and remote military operations, there is a strong requirement for solutions that provide continuous health monitoring, instant alert generation, and reliable long-range communication to ensure soldier safety and mission success [3].

Traditional soldier monitoring systems rely primarily on manual reporting or short-range communication technologies such as RF or Bluetooth [4]. These approaches are often unreliable in remote or hostile environments where network connectivity is limited or unavailable. As a result, critical health data may not reach command centres in time, delaying emergency response [5].

Additionally, conventional systems lack integration with IoT platforms, centralized monitoring, and automated alert mechanisms, making it difficult to track multiple soldiers simultaneously [6]. The absence of real-time data transmission and long-range communication further reduces the effectiveness of these systems in dynamic battlefield conditions.

In real-time scenarios, these limitations lead to several critical challenges that can compromise soldier safety and operational efficiency. Sudden changes in vital parameters such as heart rate or oxygen levels may go undetected without continuous monitoring systems. Delayed communication in remote areas can prevent timely medical assistance, increasing the risk of fatalities. Furthermore, the lack of reliable long-range communication can disrupt coordination between soldiers and command centres [7]. The absence of automated alert systems also delays response during emergencies. These challenges highlight the need for an intelligent, IoTenabled soldier monitoring system that integrates long-range communication technologies like LoRa [8], real-time health tracking, and instant alert mechanisms, ensuring enhanced safety, improved situational awareness, and efficient military operations in challenging environments.

Literature Survey

Aoun et al. [9] proposed a review of Industry 4.0 characteristics and challenges, highlighting automation, connectivity, and data-driven processes, with improvements using blockchain technology. Khan et al. [10] proposed a framework integrating blockchain, Artificial Intelligence (AI), and Industrial Internet of Things (IIoT) for digitalization of small and medium-sized enterprises. Reghunadhan [11] proposed an analysis of ethical considerations in blockchain-based systems within war zone environments using a case study approach. The study evaluated risks and ethical challenges in critical applications.

Gousteris et al. [12] proposed a secure distributed cloud storage system based on blockchain and smart contracts for data protection and integrity. Krichen et al. [13] proposed a survey of blockchain applications in modern systems, analyzing architectures, protocols, and use cases

International Journal of
AI Electronics and Nexus Energy
Peer Reviewed, Referred & Indexed Journal
ISSN: 3070-0515 www.zesterapublications.com Original Research Paper

across domains. Kassen [14] proposed the use of blockchain in e-government systems to automate public information processes and improve transparency.

Abdulrahman et al. [15] proposed a survey on the synergy between AI and blockchain in aerospace engineering, focusing on operational efficiency and technological challenges. Khoshavi et al. [16] proposed a survey of blockchain applications in transportation systems to improve operational efficiency and security. The study analysed system architectures and use cases. The work lacked real-time implementation. Lee et al. [17] proposed blockchain as a cyber defence mechanism, analysing its applications in enhancing security and resilience in digital systems.

Jain et al. [18] proposed a blockchain-based Internet of Things (IoT) architecture for drone networks, enabling secure communication and coordination. Kendall et al. [19] proposed the use of Hyperledger Fabric blockchain to improve information assurance in IoT devices for Artificial Intelligence model development. Aljohani et al. [20] proposed a study on autonomous strike unmanned aerial vehicles for homeland security missions, focusing on operational challenges and preliminary solutions.

Proposed System

Figure 1 illustrates the soldier unit (transmitter module) of the GPS-based soldier tracking and health monitoring system built around the ESP32. This unit is deployed with the soldier and is responsible for collecting real-time physiological and location data. It integrates sensors such as DHT11 for temperature and humidity, SPO2 sensor for pulse rate and oxygen level monitoring, a fall detection sensor for emergency situations, and a keypad for manual input. A GPS module provides realtime location tracking. The ESP32 processes all sensor data and transmits it wirelessly through a LoRa transmitter (Tx) to the base station. An LCD is used for local display of information.

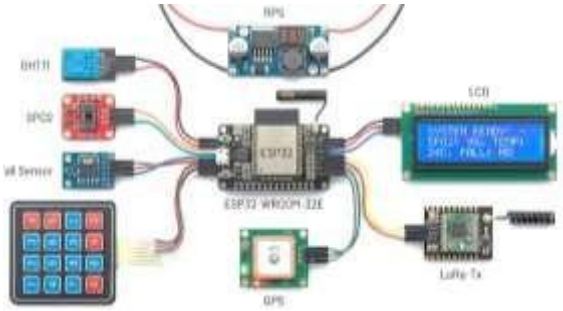


Figure 1. Soldier Unit (Transmitter) – GPSBased Health Monitoring System.

Figure 2 illustrates the base station (receiver module) of the soldier monitoring system. The ESP32 acts as the central controller, receiving data from the soldier unit via a LoRa receiver (Rx). The received data includes health parameters and GPS location. The ESP32 processes this information, displays it on an LCD, and activates a buzzer alert in case of abnormal conditions such as low oxygen levels, high temperature, or fall detection. This module ensures centralized monitoring and rapid response for soldier safety.

Step 1: Power Supply Initialization (RPS):
The regulated power supply provides stable voltage to the ESP32 and receiver components.

Step 2: Data Reception via LoRa: The LoRa receiver (Rx) collects transmitted data from the soldier unit over long distances.

International Journal of
AI Electronics and Nexus Energy
Peer Reviewed, Referred & Indexed Journal
ISSN: 3070-0515 www.zesterapublications.com Original Research Paper

Step 3: Data Processing using ESP32: The ESP32 processes received data, including health parameters and GPS location.

Step 4: LCD Display for Monitoring: The LCD displays real-time data such as temperature, SPO2 levels, and soldier location coordinates.

Step 5: Buzzer Alert System: If abnormal conditions are detected (e.g., fall detection, abnormal vitals), the buzzer is activated to alert monitoring personnel.

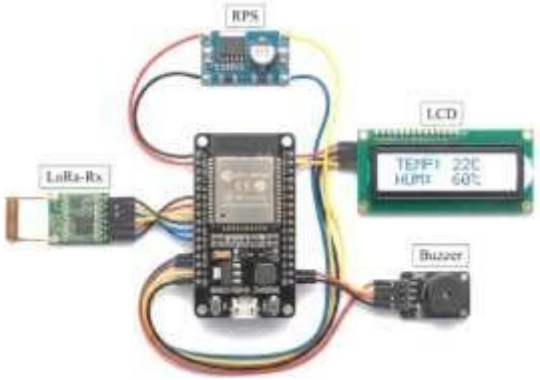


Figure 2. Base Station Unit (Receiver) – Monitoring and Alert System.

Working Procedure

The soldier unit continuously monitors health parameters and location as shown in Figure 3, processes the data using ESP32, and transmits it via LoRa to the base station. The base station receives and displays this data while generating alerts for emergencies. This system ensures real-time tracking, health monitoring, and improved safety for soldiers in critical environments.

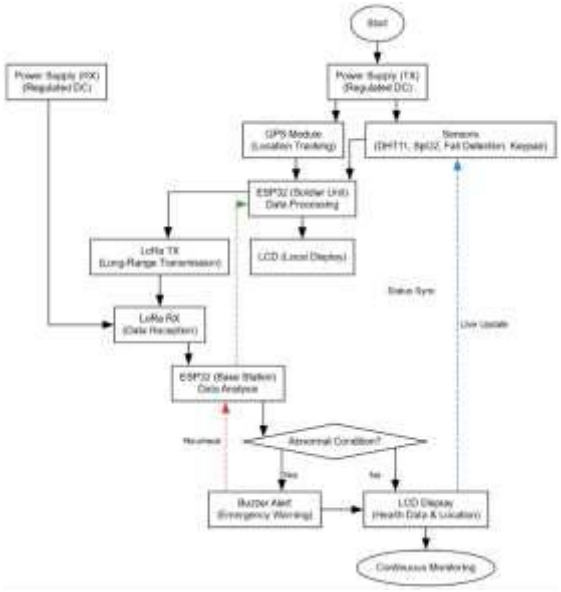


Figure 3. Proposed Flowchart.

Figure 4 illustrates the transmitter unit of a LoRa-based soldier tracking and health monitoring system designed for real-time surveillance of soldiers in remote or battlefield environments. The system is powered by a regulated power supply unit consisting of a step-down transformer, bridge rectifier, filter capacitors, and a 7805-voltage regulator to provide a stable +5V output. The ESP32 microcontroller acts as the central processing unit, interfacing with a GPS module for realtime location tracking, a SpO2 sensor for monitoring oxygen levels, a DHT11 sensor for temperature and humidity monitoring, and a vibration sensor for detecting abnormal movement or injury. A keypad is included for manual input or emergency signaling. The collected data is transmitted wirelessly using a LoRa transmitter module, while a 16×2 LCD displays real-time status. This system ensures continuous health monitoring and location tracking of soldiers in critical conditions.

International Journal of
AI Electronics and Nexus Energy
Peer Reviewed, Referred & Indexed Journal
ISSN: 3070-0515 www.zesterapublications.com Original Research Paper

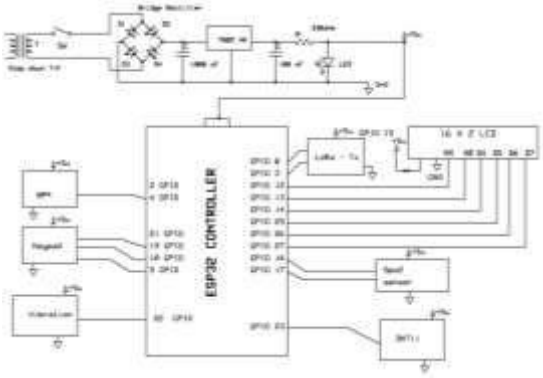


Figure 4. Transmitter Circuit Diagram.

Figure 5 presents the receiver unit of the LoRa-based soldier tracking and health monitoring system, responsible for receiving and displaying data from the transmitter. The system uses a regulated power supply like the transmitter unit. The ESP32 microcontroller receives transmitted data via a LoRa receiver module and processes it for monitoring and alert generation. An IoT module enables remote monitoring and cloud-based data access. A 16×2 LCD displays received health parameters and location information, while a buzzer provides audible alerts during emergency or abnormal conditions. This receiver system enables centralized monitoring, improving response time and ensuring the safety of soldiers in missioncritical environments.

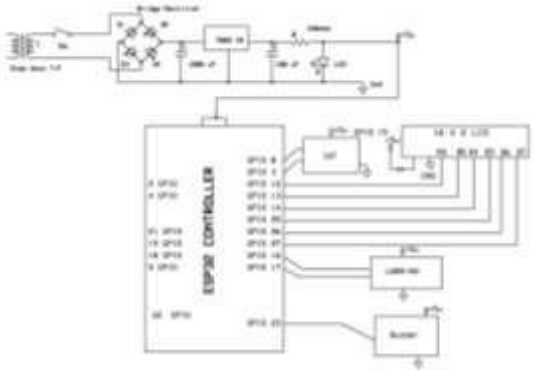


Figure 5. Receiver Unit Circuit Diagram.

Results and Discussion

Figure 6 shows the complete hardware implementation of the LoRa-based Soldier Security System consisting of both transmitter (Tx) and receiver (Rx) modules. The system integrates sensors, ESP32/Arduino controllers, LoRa communication modules, LCD displays, and buzzer units for monitoring soldier health parameters and transmitting alerts wirelessly.

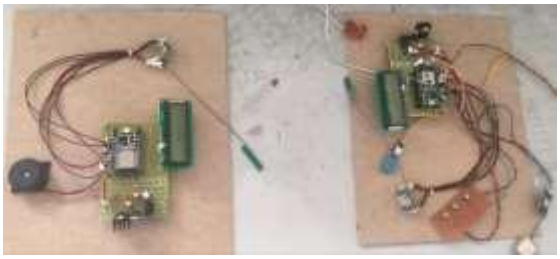


Figure 6. Hardware Setup of LoRa Based
Soldier Security System (Transmitter and
Receiver Units)

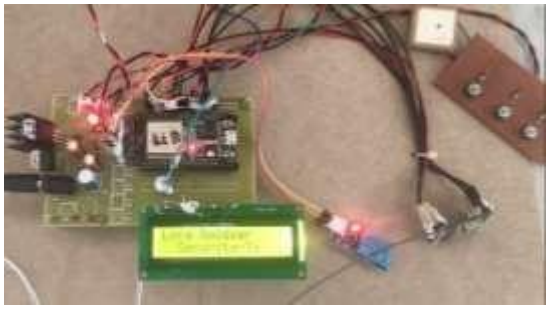


Figure 7. Transmitter (Tx) Module of LoRa
Soldier Security System

Figure 7 shows the transmitter unit carried by the soldier. It includes sensors such as SpO_2, heartbeat sensor, vibration sensor, emergency button keypad, LoRa transmitter module, LCD display, and buzzer used to collect health data and transmit alerts to the receiver unit.

International Journal of
AI Electronics and Nexus Energy
Peer Reviewed, Referred & Indexed Journal
ISSN: 3070-0515 www.zesterapublications.com Original Research Paper

Figure 8 shows the receiver unit that receives transmitted data from the soldier's transmitter module using LoRa communication. The received information is displayed on the LCD and alerts are generated using a buzzer for monitoring at the control station.

Figure 8. Receiver (Rx) Module of LoRa

Soldier Security System

Figure 9 shows the IoT web server dashboard used for remote monitoring of soldier health parameters. It displays information such as temperature, humidity, vibration status, heart rate, SpO_2 level, system status, and location along with timestamped records.

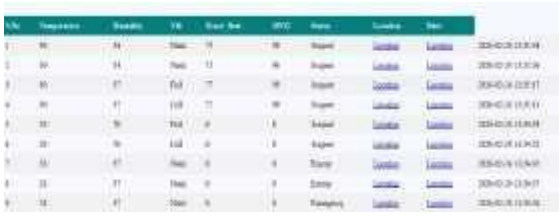


Figure 9. IoT Server Monitoring Interface.

Conclusion

The proposed LoRa-based Soldier Security System integrated with IoT offers a robust and intelligent solution for enhancing soldier safety in modern battlefield environments by enabling long-range communication, real-time health monitoring, and automated alert mechanisms. By incorporating SpO_2 and heartbeat sensors, the system continuously tracks vital parameters, allowing early detection of critical health conditions. The use of LoRa communication ensures reliable, lowpower, and long-distance data transmission even in remote or hostile areas where conventional networks may fail. The inclusion of an emergency keypad provides manual alert capability, while buzzer and LCD notifications enable immediate local awareness.

Additionally, IoT integration facilitates realtime data transmission to command centers, improving situational awareness and enabling rapid decision-making. This system effectively overcomes the limitations of traditional monitoring approaches by providing continuous, automated, and reliable communication.

References

[1] Davis, S.I. Artificial intelligence at the operational level of war. Def. Secur. Anal. 2022, 38, 74–90.

[2] Mattingsdal, J.; Johnsen, B.H.; Espevik, R. Effect of changing threat conditions on police and military commanders' preferences for urgent and offensive actions: An analysis of decision-making at the operational level of war. Mil. Psychol. 2023, 37, 33–49.

[3] Mattingsdal, J.; Espevik, R.; Johnsen, B.H.; Hystad, S. Exploring why police and military commanders do what they do: An

empirical analysis of decision-making in hybrid warfare. Armed Forces Soc. 2024, 50, 1218– 1244.

[4] Explainable AI Framework for PolicyCompliant Anomaly Detection in Data Pipelines. (2025). Inaternational Journal of Communication Networks and Information Security, 16(4). https://doi.org/10.48047/ijcnis.16.4.21
11

[5] Gaddam, S. Integrating Analytics into the Development Process: Bridging the Gap between Data Insights and Design Execution.

[6] [6] Prodduturi, S. M. K. To Secure Your Paper as Per UGC Guidelines We Are Providing A Electronic Bar code.

[7] Reddy, S. K. R. (2025). Tailoring Loyalty Rewards Systems across Industries: Cloud vs On-Prem Solutions. International Journal of All Research Education and Scientific Methods (IJARESM).

[8] Poojari, R. (2025). A Comparative Analysis of Fine-Tuning Versus Retrieval-Augmented Approaches for Enhancing Healthcare-Centric Large Language Models.

[9] Kalae, U. K. (2025). Optimizing costeffective cloud data pipeline orchestration across multiple cloud providers. Journal of Information Systems Engineering

[10] Khan, A.A.; Laghari, A.A.; Li, P.; Dootio, M.A.; Karim, S. The collaborative role of blockchain, artificial intelligence, and industrial internet of things in digitalization of small and medium-size enterprises. Sci. Rep. 2023, 13, 1656.

[11] Reghunadhan, R. Ethical considerations and issues of blockchain technology-based systems in war zones: A case study approach.
In Handbook of Research on Blockchain Technology; Academic Press: Cambridge, MA, USA, 2020; pp. 1–34.

[12] Gousteris, S.; Stamatiou, Y.C.; Halkiopoulos, C.; Antonopoulou, H.; Kostopoulos, N. Secure Distributed Cloud Storage based on the Blockchain Technology and Smart Contracts. Emerg. Sci. J. 2023, 7, 469–479.

[13] Krichen, M.; Ammi, M.; Mihoub, A.; Almutiq, M. Blockchain for modern applications: A survey. Sensors 2022, 22, 5274.

[14] Kassen, M. Blockchain and egovernment innovation: Automation

International Journal of
AI Electronics and Nexus Energy
Peer Reviewed, Referred & Indexed Journal
ISSN: 3070-0515 www.zesterapublications.com Original Research Paper

of public information processes. Inf. Syst. 2022, 103, 101862.

[15]Abdulrahman, Y.; Arnautović, E.; Parezanović, V.; Svetinovic, D. AI and blockchain synergy in aerospace engineering: An impact survey on operational efficiency and technological challenges. IEEEAccess 2023, 11, 87790–87804.

[16] Khoshavi, N.; Tristani, G.; Sargolzaei, A. Blockchain applications to improve operation and security of transportation systems: A survey. Electronics 2021, 10, 629. [17] Vasagam, M., Kumar, A., & Garg, A. (2026). Learning Execution Plan Embeddings for Multi-Dimensional Query Resource Prediction. IEEE Access.

[18] Patyrykin, K.(2025). CANCEL CULTURE PROBLEM. Lex Localis:

Journal of Local Self-Government, [19] Purmani, S. S. R. (2025). Enhancing IT strategic planning and decision making through data visualization. International Journal of Enhanced Research in Management & Computer Applications, 14(4), 75–81 [20] Kumara, S. (2026, February). A Lightweight Deep Learning Based Classification Models for Non-Human Identity Threat Detection. In 2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC) (pp. 1-6). IEEE.